

A Policy-Indexed Method for Canonical Reconstruction and Fail-Closed Delivery of Untrusted Media

Oleksandr Melnychenko ^{*}

Khmelnytskyi National University, Khmelnytskyi, Ukraine

Abstract

Authenticated channels may deliver ambiguous media unsuitable for local use. We define a transport-independent method that reconciles filename, declared type, and bounded byte evidence; applies policy-required reconstruction; validates the candidate independently; and publishes only clean, non-overwriting output. The method addresses type confusion, ambiguous structures, retained codec payloads, resource amplification, and publication faults across image, audio, ISO base media, and WebM profiles. The analysis gives conditional canonical-delivery and fixed-witness post-reconstruction monotonicity theorems under authorizing-parser soundness. A bounded experiment used 104 generated benign or inert-unsupported records. All fixture-derived action oracles matched, measuring fixture-policy consistency rather than independent ground truth. Eighty accepted evaluations produced 40 unique outputs that passed at least two applicable external checks; 11 selected fault checks recorded no publication-invariant violation, and 7,299,439 shortened-schedule fuzz executions produced no recorded crash. Fidelity passed for 40/80 accepted records; all audio and source-WebM evaluations failed. A release-ineligible pilot accepted 10 public benign files (four exact-idempotent and six lossy closures), while discovery retained 316 distinct candidates. Mechanical identity checks passed, but unsigned review forced fail-closed selection of 0/246 sources. Missing adversarial strata, independent audit, admission, and controlled schedules authorize no release, population, malware-detection, or capacity claims.

Keywords: content disarm and reconstruction, canonical parsing, fail-closed publication, untrusted media, polyglot files, media security

^{*}Corresponding author.

Email address: `melnychenko@khmnu.edu.ua` (Oleksandr Melnychenko )

1. Introduction

Digital media cross local trust boundaries through email, web download, cloud synchronization, direct import, and messaging. Upstream controls can establish channel authenticity, access rights, or transport confidentiality without establishing that the received octet string is an admissible input for a local parser, decoder, preview generator, or file-opening application. A file can arrive through an authenticated channel and still exploit a decoder, preserve privacy-bearing metadata, conceal an active object, amplify resource demand, or present different meanings to different parsers. End-to-end encrypted messaging is one relevant instance because a blind relay cannot inspect plaintext without changing the confidentiality model [1, 2, 3]. The underlying file-admission problem is not specific to that transport.

Conventional file controls do not fully settle the problem. A filename suffix and a caller-supplied media type are declarations rather than proof of a complete file grammar. A magic-byte recognizer can identify a plausible prefix while leaving a trailing object unaccounted for. A malware signature can match a known byte pattern but cannot establish that every accepted output belongs to one intended format or that all excluded metadata and structures were removed. A successful external transcoder exit establishes only that one tool produced bytes. It does not establish that the bytes satisfy the local canonical output policy. These mechanisms remain useful evidence, but none of them alone authorizes publication to the endpoint filesystem or renderer.

Content disarm and reconstruction methods address part of this problem by replacing an input with a reduced representation. Existing work has established the importance of parser agreement, complete input consumption, generated parsers, and canonical output languages [4, 5, 6, 7, 8, 9, 10]. This paper addresses their composition at a local publication boundary. Classification, transformation, authorization, and filesystem publication remain separate decisions, and the claimed reconstruction strength records whether compressed source payloads were retained or replaced from decoded pixels, frames, or samples.

This study develops a transport-agnostic method for policy-indexed canonical media reconstruction and fail-closed publication. The protected transition is from an untrusted media tuple to either a policy-admissible reconstructed object or no deliverable bytes. The method first converges evidence from the portable name, supplied media type, and bounded byte recognizer. It then selects one source profile, performs the reconstruction level required by policy, subjects the candidate to an authorizing parser on a code path separate from reconstruction, applies post-rebuild signature and

resource policy, and invokes publication only after a clean verdict. Delivery therefore depends jointly on one source route, a registered source-to-output mapping, policy-indexed canonicity, reconstruction-level sufficiency, output scanning, resource bounds, and non-overwriting publication. Every other result exposes no deliverable bytes, creates no new target through the component interface, and leaves any pre-existing target unchanged.

The claim has separate analytical and empirical falsifiers. Analytically, under the registered-parser and filesystem assumptions, every delivered byte string must belong to one policy-indexed canonical language, and every non-clean terminal state must expose no deliverable bytes. Empirical conformance requires each accepted manifest record to satisfy its prespecified action, reason-family, pipeline-state, reconstruction, closure, and resource oracles, together with the executed fault schedule. One violated obligation falsifies conformance for that run. Fidelity is evaluated independently and cannot authorize a structurally invalid output. The contribution evaluated here is the policy-indexed authorization relation that makes route selection, source-to-output conversion, reconstruction strength, output validation, and publication jointly auditable.

Research question 1 (RQ1) asks when heterogeneous endpoint evidence selects exactly one admissible route. RQ2 asks which conditions are sufficient to restrict canonical delivery to clean verdicts at each reconstruction level. RQ3 asks whether the implementation preserves those conditions on a frozen corpus under malformed and ambiguous carriers, decoder and publication faults, cancellation, and fixed resource limits, and what fidelity and latency costs accompany the observed paths. The first two questions receive analytical answers. RQ3 is only partially exercised by the bounded finite-corpus experiment and remains unanswered at its declared controlled and population scope.

The scope is limited to measurable media-carrier properties. Malware family and operational role require behavioral evidence that media bytes alone do not provide, as detailed in Section 2. The method also does not remove harmful visual or auditory meaning, prove the absence of steganography, compensate for a compromised endpoint, or make an unsafe decoder trustworthy. For a frozen set of supported profiles and explicit assumptions, it restricts publication to canonical outputs validated separately from their construction and produced at or above the policy-required reconstruction level.

2. Background, Terminology, and Research Position

Early malware terminology separates self-replication from host dependence and propagation [11, 12]. Contemporary operational labels add initial-access mechanism, staged-delivery role, payload objective, persistence, and privacy impact [13, 14, 15]. These distinctions guide corpus design but are not interchangeable with carrier structure. A media file may carry an exploit trigger without being a self-replicating program. A downloader may arrive through a document, archive, executable, or a parser flaw in nominal media. Conversely, suspicious metadata or an ambiguous container does not establish any malware family. This study therefore reports measurable carrier properties and keeps behavioral attribution outside the acceptance decision.

Delivery and storage systems establish different upstream properties: sender or server authentication, access control, transport integrity, provenance, or confidentiality. None of these properties entails whole-file grammar, canonicity, or safe publication. In end-to-end encrypted messaging the inspection point must remain local because moving plaintext inspection to a relay would alter the confidentiality model [1, 2, 3]. Email, cloud synchronization, web download, and direct import reach the same local admission problem through different upstream assumptions. The proposed method therefore begins at the media trust boundary and treats transport provenance as context rather than acceptance evidence.

Parser-differential research shows why one recognizer is insufficient. Different tools can disagree about valid prefixes, terminal offsets, duplicated fields, integer interpretation, and nested structure [5, 6]. Work on canonical parsing, language-oriented security, and generated parsers replaces permissive recognition with an explicit input language and complete parsing [4, 7, 8, 9]. SafeDocs applies the related principle that complex data should be reduced to well-defined, safely processable forms [10]. Recent studies of file ambiguity and polyglots further demonstrate that extension, media type, and isolated parser success do not determine one whole-file meaning [16, 17, 18, 19].

Content disarm and reconstruction operationalizes reduction by deleting excluded structures or by decoding and re-encoding content. The relevant security claim depends on what is retained. Structural reconstruction parses the source and serializes an allowed structure while retaining selected compressed payloads. Semantic reconstruction derives a new representation from decoded pixels, samples, frames, or equivalent media semantics. Structural reconstruction can remove metadata, normalize layout, and exclude unaccounted suffixes. It cannot support a claim that retained compressed payload bytes have been neutralized. Semantic reconstruction supports a

stronger source-independence claim, but only under explicit decoder, encoder, resource, and fidelity assumptions.

Let the attacker-influenced request evidence be $r = (x, n, m)$, where x is the input octet string, n is one portable display basename, and m is the supplied media type. Let $c = (p, d)$ be trusted processing context, where p is an immutable policy snapshot and d contains the enforced deadline and cancellation state. One processing request is therefore

$$u = (r, c) = ((x, n, m), (p, d)). \quad (1)$$

An untrusted caller may request earlier cancellation or a shorter deadline but cannot weaken p or extend the host-imposed processing budget. The request excludes a caller-controlled output path. A final path is derived only after acceptance from a policy-controlled directory, a sanitized basename, and the canonical output suffix.

A format profile is more specific than a broad media family. A source profile s fixes the admissible input evidence and reconstruction route. An output profile k fixes a canonical output grammar, media type, suffix set, resource limits, and parser version. Policy also registers the permitted source-to-output mappings and the minimum reconstruction level. This distinction is necessary because strict routes may convert WAV, FLAC, or Ogg input to MP3, and WebM input to MP4. Static and animated WebP are separate profiles because their component languages and reconstruction claims differ. Likewise, WebM and general Matroska cannot share one native structural route merely because both use EBML.

For policy p and profile k , let $L_{p,k} \subseteq \{0, 1\}^*$ denote the canonical output language. A sound authorizing parser starts at offset zero, terminates at $|y|$, admits every component at its parsed location, and checks ordering, cardinality, cross-reference, integrity, and resource constraints. If $\rho_{p,v,k}(y)$ is its report under policy p and parser-registry version v , the soundness direction used here is

$$\text{CompleteOK}_p(\rho_{p,v,k}(y), k, y) \implies y \in L_{p,k}. \quad (2)$$

The reverse implication is not assumed: a conservative parser may reject a language member. A successful prefix parse is not complete acceptance, and membership is not permanent across policy or parser-registry versions. Canonicity always refers to a named profile and frozen policy.

Reconstruction levels form the ordered set

$$\text{NotProcessed} \prec \text{Structural} \prec \text{Semantic}. \quad (3)$$

The order describes admissible claim strength, not a universal quality rank-

ing. A policy requirement $q_{p,s}$ for source profile s is satisfied only when the performed level obeys $\ell \geq q_{p,s}$. Unsupported input is blocked rather than misrepresented as structurally reconstructed.

Verdicts are separated from pipeline errors:

$$V = \{\text{Clean}, \text{Suspicious}, \text{Blocked}\}, \quad O = \text{Result}(V \times E, \text{PipelineError}), \quad (4)$$

where E contains bounded stage evidence. In enforcement mode, a clean verdict means that every mandatory check succeeded and no policy warning or blocker remains. A suspicious verdict means that no blocking condition remains but at least one policy warning does. A blocked verdict records at least one blocking condition. Suspicious and blocked results are both non-deliverable. A pipeline error is also non-deliverable but remains distinct for diagnosis and availability analysis. These cases are mutually exclusive, and no failure is converted into a clean result by exception handling or fallback routing.

An authorizing parser is considered separate from construction when it is invoked after reconstruction through a code path distinct from the handler’s source parser and writer. This separation does not imply that the parser was independently verified. The stronger term *implementation-diverse validator* is reserved for external parser or decoder families used in the differential evaluation. The distinction prevents separation within one implementation from being reported as independent replication.

Table 1 locates the proposed method at the intersection of these lines of work. It combines routing to exactly one admissible profile, explicit reconstruction strength, separate output validation, and publication restricted to clean verdicts, with a distinct falsification oracle for each obligation.

3. Threat Surface and System Context

The method is placed wherever externally influenced media becomes eligible for local use. The upstream source may be an authenticated message, mail attachment, cloud-synchronized object, browser download, removable medium, or direct import. The downstream consumer may be durable storage, a preview generator, a media decoder, an export interface, or another application. Upstream provenance does not substitute for validation of the exact octet string crossing this local trust boundary.

In an end-to-end encrypted application, this boundary lies after authenticated decryption and before local storage or rendering. The relay can remain blind to plaintext and the local verdict. In other deployments the same method begins after download or import. It neither establishes transport

Table 1: Research position relative to selected technical lines of work.

Line of work	Established contribution	Boundary addressed in this work
Signature and malware scanning [13]	Recognition of configured indicators and known malicious structures	Does not establish complete grammar, reconstruction strength, or publication safety
Canonical and verified parsing [4, 7, 8, 9]	Grammar-defined recognition, complete parsing, and reduced parser ambiguity	Joint route, reconstruction-strength, output-validation, and publication judgment
Content disarm and reconstruction [10, 17, 20]	Format-specific transformation into reduced representations	Policy-indexed claim strength and authorization of the emitted object rather than handler success
Parser differentials and polyglots [5, 18, 19]	Detection and characterization of conflicting file interpretations	Routing only when the evidence admits one profile, followed by canonical output observation and fail-closed delivery

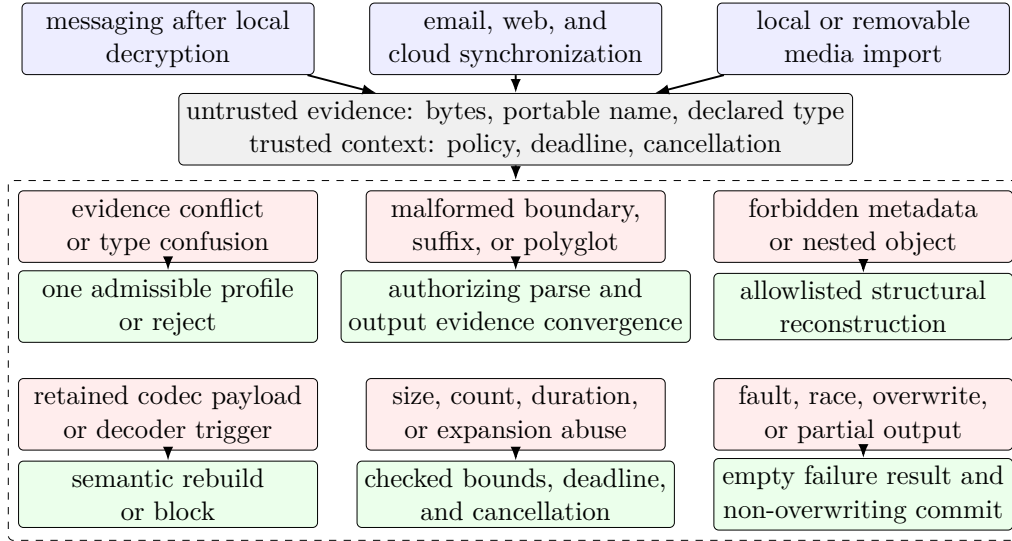


Figure 1: Source-independent threat-control map. Upstream channel properties do not authorize media bytes. Each measurable carrier or publication threat is assigned a distinct fail-closed control.

confidentiality nor changes the authentication model of the source channel.

The protected assets are parser and client availability, privacy-sensitive metadata, integrity of reconstructed content, policy consistency, and integrity

of the final publication transition. Endpoint execution integrity is an asset of the enclosing deployment but is not guaranteed against a compromised external decoder by this component alone. The adversary controls the full input byte string, display name, supplied media type, internal sizes and offsets, metadata, compressed payload, dimensions, counts, duration fields, and submission timing. The adversary may construct malformed files, prefix and suffix camouflage, whole-file polyglots, nested objects, decompression amplification, decoder regressions, and cancellation races. Repeated submissions and adaptive selection based on visible success or failure are permitted. The policy snapshot is trusted. Caller cancellation and earlier deadlines may shorten work but cannot enlarge the authorized policy or the host processing budget.

The in-scope threat classes are covered by

$$\mathcal{T}_{\text{in}} = \mathcal{T}_{\text{id}} \cup \mathcal{T}_{\text{grammar}} \cup \mathcal{T}_{\text{payload}} \cup \mathcal{T}_{\text{resource}} \cup \mathcal{T}_{\text{publication}}, \quad (5)$$

where the classes denote evidence and type confusion, malformed or ambiguous whole-file structure, retained or forbidden payload structure, computational amplification, and fault-driven publication effects. The classes may overlap. A test record may carry secondary tags, while its prespecified primary class determines the aggregate denominator and primary falsification oracle. Arbitrary decoder compromise is a conditional deployment threat because process supervision alone does not isolate same-user host effects. Harmful decoded meaning and steganography are content-analysis problems outside $\mathcal{T}_{\text{in}}$.

Malware labels organize scenario provenance but do not enter the output verdict. The behavioral distinctions defined in Section 2 prevent a carrier property from being reported as evidence of replication, delivery role, or payload objective. The empirical corpus can therefore contain inert carriers shaped around those mechanisms without implying a malware-family detection rate.

The trusted computing base includes the authenticated application binary, the loaded immutable policy snapshot, the registered authorizing parsers, trusted provenance records, and the operating-system enforcement of process and filesystem primitives. Collision-resistant digests identify and integrity-check recorded artifacts but do not authenticate their provenance by themselves. The endpoint is assumed uncompromised before processing begins. Memory-safety guarantees reduce one class of implementation defects but do not prove parser correctness. Foreign code and external media tools remain potentially faulty. Their output is not deliverable until an authorizing parser accepts the complete object. Process supervision

Table 2: Adversary capabilities and the corresponding enforced boundary.

Control	Security-relevant effect	Required response
Name and supplied type	Misrouting, hidden suffix, or type confusion	Portable-name grammar and evidence convergence before dispatch
Arbitrary byte layout	Prefix success, trailing object, overlap, or malformed boundary	Offset-zero complete parsing and exact terminal offset
Metadata and container structure	Privacy leakage, nested active object, or stale reference	Location-aware allowlist and reconstruction
Compressed payload	Exploitation during source decoding or retention of a source representation in the result	Isolate or trust the decoder for transformation-time risk. Use semantic reconstruction when source-payload retention is prohibited
Counts, dimensions, and timing	Allocation, expansion, or execution exhaustion	Checked arithmetic, limits, deadline, and cancellation
Fault timing and destination state	Partial publication or overwrite	Temporary output, validation before commit, and non-overwriting persistence with deployment-verified atomic visibility

bounds selected liveness, output, and publication effects. It does not confine arbitrary same-user filesystem or host effects after decoder code execution. Deployments that include decoder compromise in scope must add a platform sandbox such as a dedicated account, namespace and system-call policy, container or mandatory-access-control profile, or application sandbox.

The method assigns five responsibilities to distinct stages: evidence convergence and routing, profile-specific reconstruction, complete validation and rebuilt-output scanning, process supervision, and final result construction with publication. Candidate-producing stages do not receive authority to publish their own output. The concrete module and interface realization is described in Section 8.

The method assumes an uncompromised endpoint and does not verify the external codec, operating system, filesystem, or hardware. Its media-only policy excludes active documents, archives, executables, scripts, PDF,

and vector markup. Accepted visual or auditory content may still be harmful, misleading, age-inappropriate, or steganographic. Per-request limits also do not replace system-wide admission control and scheduling.

The eight assurance obligations are complete output consumption (F1), component closure (F2), output evidence convergence (F3), semantic source-independence (F4), second-pass stability (F5), bounded execution (F6), validation separate from construction (F7), and clean-result publication with fault containment (F8). Each has a separate falsification condition in the evaluation protocol, and failure in one obligation cannot be offset by success in another.

4. Canonical Reconstruction and Publication Method

The method separates source routing from output authorization. For source profile s , let $E_s(n)$ denote evidence derived from the normalized final suffix, $M_s(m)$ evidence derived from the supplied media type, and $B_s(x)$ the result of a bounded signature and structural-header probe. These functions may narrow the eligible source profiles but do not establish that the whole input belongs to any profile. Their conjunction defines

$$R_p(x, n, m) = \{s \in K_p^{\text{in}} : E_s(n) \wedge M_s(m) \wedge B_s(x) \wedge C_{p,s}(n, m)\}, \quad (6)$$

where K_p^{in} is the finite enabled source-profile set and $C_{p,s}$ contains explicit alias and compatibility rules. Routing proceeds only when $|R_p(x, n, m)| = 1$. An empty set denotes unsupported or conflicting input, whereas a set with more than one member denotes ambiguity. Neither case is resolved through precedence, extension override, or best-effort fallback.

Normalization accepts one portable basename and one syntactically valid media type. It rejects path separators, dot components, control or bidirectional characters, forbidden device names, trailing dots or spaces, wildcard types, malformed types, and dangerous suffix chains. Alias equivalence is registered by profile. For example, `audio/mp4`, `audio/x-m4a`, and `audio/m4a` can converge on the same M4A profile. An alias does not broaden byte acceptance or authorize another ISO base media profile. HTTP content-type semantics likewise do not turn a supplied declaration into whole-file evidence [21].

For the unique source route s , reconstruction is a total typed operation

$$\text{Rec}_{p,s}(u) \in \text{Result}(\mathcal{A}, \text{PipelineError}), \quad a = (y, n_o, m_o, k_o, \ell, e) \in \mathcal{A}, \quad (7)$$

where y is the candidate byte string, n_o is its generated portable name, m_o is its declared output media type, k_o is its output profile, ℓ is the performed

reconstruction level, and e is bounded stage evidence. Failure returns a typed non-deliverable error. A successful operation returns $\text{Ok}(a)$, not a final verdict, so transformation success is not confused with authorization.

The performed level records how the output was obtained. Structural output is serialized from a complete parsed source using an allowlisted component grammar and may retain selected compressed payload. Semantic output is serialized from a decoded intermediate representation under policy bounds. Let $\Omega_p \subseteq K_p^{\text{in}} \times \{\text{NotProcessed}, \text{Structural}, \text{Semantic}\} \times K_p^{\text{out}}$ be the registered source-level-output relation. It permits format-preserving routes and explicit conversions such as WAV to MP3 or WebM to MP4. The required level $q_{p,s}$ is fixed before dispatch. Acceptance requires

$$\ell \geq q_{p,s} \quad \wedge \quad (s, \ell, k_o) \in \Omega_p. \quad (8)$$

A handler cannot lower this requirement based on input convenience. If a semantic path is unavailable or fails, a strict semantic profile blocks instead of returning a structural candidate.

The versioned registry Π_v contains authorizing parsers whose code paths are separate from the constructing handlers. Each output profile k names a nonempty configured set $\Pi_{p,v,k}^{\text{auth}} \subseteq \Pi_v$. A parser report ρ records the initial offset $\rho.o$, terminal offset $\rho.t$, observed profile, component count, canonicity checks, and bounded observations. Define

$$\begin{aligned} \text{CompleteOK}_p(\rho, k, y) \equiv & \rho.\text{ok} \wedge \rho.o = 0 \wedge \rho.t = |y| \\ & \wedge \rho.\text{profile} = k \wedge \rho.\text{canonical}_{p,k}, \end{aligned} \quad (9)$$

where $\rho.\text{canonical}_{p,k}$ denotes satisfaction of the policy-indexed component, ordering, cardinality, integrity, cross-reference, and resource predicates for $L_{p,k}$. The configured authorizing condition is

$$\text{Valid}_{p,v,k}(y) \equiv \Pi_{p,v,k}^{\text{auth}} \neq \emptyset \wedge \bigwedge_{j \in \Pi_{p,v,k}^{\text{auth}}} \text{CompleteOK}_p(\rho_j(y), k, y). \quad (10)$$

The reference implementation normally selects one registered authorizing grammar from the declared output type and then checks that its report agrees with the generated name and byte recognizer. It does not run every parser in the registry. External validators from different tool lineages are used only in the empirical protocol and do not enter the analytical acceptance predicate.

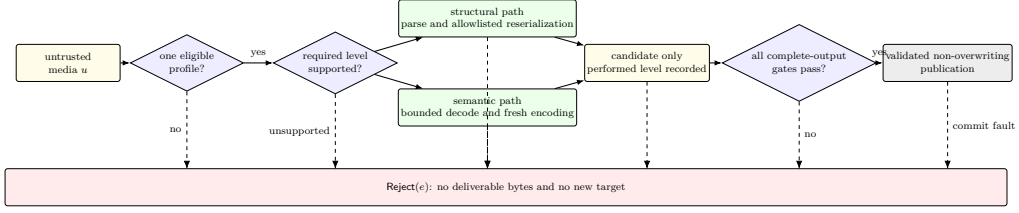


Figure 2: Fail-closed reconstruction decision. Solid edges form the only publication path. Dashed edges denote disagreement, unsupported reconstruction, transformation failure, failed or indeterminate authorization, and commit failure. All converge on one non-deliverable terminal state.

The candidate acceptance predicate is

$$\begin{aligned}
 \text{Accept}_{p,v}(u, a) \equiv & R_p(x, n, m) = \{s\} \wedge \text{Rec}_{p,s}(u) = \text{Ok}(a) \\
 & \wedge \text{InputOK}_p(n, m, s) \wedge (s, \ell, k_o) \in \Omega_p \\
 & \wedge \text{NameOutOK}_p(n_o, k_o) \wedge \text{TypeOutOK}_p(m_o, k_o) \\
 & \wedge \text{ByteTypeOK}_p(y, k_o) \wedge \text{LevelOK}_p(s, \ell) \\
 & \wedge \text{BoundsOK}_p(x, y, e) \\
 & \wedge \text{ScanOK}_p(y) \wedge \text{Valid}_{p,v,k_o}(y).
 \end{aligned} \tag{11}$$

Every conjunct is necessary. Signature policy is evaluated on the reconstructed bytes because scanning only the source would not authorize the emitted object. An optional pre-scan can reject early but cannot replace the rebuilt-output scan. Output bytes, expansion ratio, dimensions, frames, duration, channels, component counts, and nesting are checked with overflow-safe arithmetic.

Reconstruction success creates only a candidate. An unavailable required level cannot become a weaker pass-through path, and bytes remain unauthorized until complete output validation. Figure 2 summarizes this decision.

The eight analytical invariants are summarized in Table 3. Each specifies an observable condition that maps directly to a test or release gate.

Policy comparison is defined *syntactically* on the components that enter post-reconstruction validation, so that the monotonicity result below is a theorem about those components rather than a restatement of an extensional inclusion. Let $w = (s, a, \{\rho_j(y)\})$ be a fixed post-reconstruction witness, with $a = (y, n_o, m_o, k_o, \ell, e)$, and let $\text{PostOK}_{p,v}(w)$ denote the conjunction of the source-level-output relation and the output-name, output-type, byte-type, level, bounds, scan, and authorizing-parser predicates in Equation (11) (that is, $\text{Accept}_{p,v}$ with the routing conjunct $R_p(x, n, m) = \{s\}$ removed).

Definition 4.1 (Syntactic policy refinement). *Write $p_2 \sqsubseteq p_1$ when, for*

Table 3: Formal invariants and their empirical interpretation.

ID	Invariant	Required observation
F1	Complete output consumption	Parser starts at zero and terminates at $ y $
F2	Component closure	Every emitted component is admitted at its parsed location
F3	Output evidence convergence	Generated name, declared type, byte recognition, and the authorizing report identify one output profile
F4	Semantic source-independence	No source payload is copied or referenced as an output component on a strict semantic route
F5	Second-pass stability	A second pass is exact for exact profiles or satisfies the registered semantic closure for lossy profiles
F6	Bounded execution	Bytes, structure, time, expansion, and child resources remain within policy
F7	Validation separate from construction	The authorizing parser is distinct from the constructing handler
F8	Clean-result publication	Non-clean and error outcomes expose no bytes, create no new target, and leave any pre-existing target unchanged

the fixed output profile k_o and parser-registry version v , all of the following hold componentwise: (i) every scalar bound of p_2 is no larger than the corresponding bound of p_1 , i.e. $\mathcal{B}_{p_2} \leq \mathcal{B}_{p_1}$ coordinatewise; (ii) the required reconstruction level is no weaker, $q_{p_2,s} \geq q_{p_1,s}$, and $\Omega_{p_2} \subseteq \Omega_{p_1}$; (iii) the output language is no broader, $L_{p_2,k_o} \subseteq L_{p_1,k_o}$, together with the corresponding inclusions for the admissible name and media-type sets, $\mathcal{N}_{p_2,k_o} \subseteq \mathcal{N}_{p_1,k_o}$ and $\mathcal{M}_{p_2,k_o} \subseteq \mathcal{M}_{p_1,k_o}$; (iv) the signature set is no smaller, $\Sigma_{p_1} \subseteq \Sigma_{p_2}$, so that p_2 scans for at least the indicators p_1 does and admits no scan bypass; and (v) the authorizing-parser set and canonicity predicate are preserved, $\Pi_{p_1,v,k_o}^{\text{auth}} = \Pi_{p_2,v,k_o}^{\text{auth}}$ and $\rho.\text{canonical}_{p_1,k_o} \Leftrightarrow \rho.\text{canonical}_{p_2,k_o}$, and the publication contract of Equation (14) is unchanged.

Note the deliberate direction of the signature-set inclusion in Definition 4.1(iv). The scan predicate $\text{ScanOK}_p(y)$ holds when no signature in Σ_p matches y ; it is therefore *antimonotone* in the signature set, and a naive “larger Σ ” ordering would break the result. Requiring $\Sigma_{p_1} \subseteq \Sigma_{p_2}$ is what makes ScanOK monotone in the same direction as the other conjuncts.

Lemma 4.2 (Componentwise monotonicity). *If $p_2 \sqsubseteq p_1$ then each con-*

conjunct of PostOK is preserved from p_2 to p_1 on the fixed witness w : for every component predicate π among the source-level-output, output-name, output-type, byte-type, level, bounds, scan, and authorizing-parser checks, $\pi_{p_2}(w) \Rightarrow \pi_{p_1}(w)$.

Proof. Each conjunct is checked against a set or bound that Definition 4.1 relaxes from p_2 to p_1 . *Bounds:* **BoundsOK** asserts that the recorded scalars of (y, e) lie at or below the policy bounds; by (i) the p_1 bounds dominate the p_2 bounds, so satisfaction under p_2 implies satisfaction under p_1 . *Level:* **LevelOK** asserts $\ell \geq q_{p,s}$ and $(s, \ell, k_o) \in \Omega_p$; by (ii) $q_{p_1,s} \leq q_{p_2,s} \leq \ell$ and $\Omega_{p_2} \subseteq \Omega_{p_1}$. *Output language, name, and type:* **ByteTypeOK**, **NameOutOK**, and **TypeOutOK** assert membership of y , n_o , and m_o in the respective admissible sets; by (iii) each p_2 set is a subset of the corresponding p_1 set, so membership is preserved. *Scan:* **ScanOK_p**(y) holds when Σ_p has no match on y ; by (iv) $\Sigma_{p_1} \subseteq \Sigma_{p_2}$, so the absence of a Σ_{p_2} -match implies the absence of a Σ_{p_1} -match. *Authorizing parser:* by (v) the authorizing set and canonicity predicate are identical under p_1 and p_2 , and the reports $\{\rho_j(y)\}$ are fixed in w , so $\text{Valid}_{p_2,v,k_o}(y) \Leftrightarrow \text{Valid}_{p_1,v,k_o}(y)$. Each conjunct is therefore preserved. $\square$

Theorem 4.3 (Post-reconstruction validation monotonicity). *For a fixed post-reconstruction witness w and parser-registry version v ,*

$$p_2 \sqsubseteq p_1 \wedge \text{PostOK}_{p_2,v}(w) \implies \text{PostOK}_{p_1,v}(w). \quad (12)$$

Proof. $\text{PostOK}_{p,v}(w)$ is the conjunction of the component predicates in Lemma 4.2. Assuming $p_2 \sqsubseteq p_1$ and $\text{PostOK}_{p_2,v}(w)$, every p_2 -conjunct holds; by Lemma 4.2 each corresponding p_1 -conjunct holds; their conjunction is $\text{PostOK}_{p_1,v}(w)$. $\square$

Theorem 4.3 is a post-reconstruction validation property, not an end-to-end routing property, and it holds only with the witness w (in particular the candidate y and the reports $\{\rho_j(y)\}$) held fixed. A policy change can alter the eligible route set, select a different reconstruction, or produce different bytes. Such cases are recorded as route or reconstruction changes and do not instantiate Equation (12).

The result constructor enforces the delivery boundary with an option type:

$$\text{deliverable}(o, u, a) = \begin{cases} \text{Some}(y), & o = \text{Ok}(\text{Clean}, e) \wedge \text{Accept}_{p,v}(u, a), \\ \text{None}, & \text{otherwise.} \end{cases} \quad (13)$$

This rule applies to in-process values, interface buffers, and path publication. Suspicious, blocked, cancelled, timed-out, malformed, unsupported, and in-

ternally failed operations are distinguishable in evidence while sharing the same no-deliverable postcondition.

Path publication has a non-overwriting abstract contract. The system derives the final basename and canonical suffix, stages accepted bytes, revalidates the policy snapshot and destination condition, and invokes a publication primitive whose platform-specific guarantees are stated separately. The abstract transition is

$$(D, \perp) \xrightarrow{o=\text{Ok}(\text{Clean}, e) \wedge \text{Accept}_{p,v}(u, a)} (D \cup \{q \mapsto y\}, q), \quad (14)$$

where D is the pre-state and $q \notin \text{dom}(D)$. A failed component publication call neither creates nor replaces q . If q already exists or is created by another actor during the attempt, that entry remains unchanged. The component does not report a staging object as a final path and removes staging state on abort to the extent guaranteed by the host filesystem.

The delivery guarantee rests on three assumptions that are stated explicitly because they, not the acceptance predicate, carry the security weight.

Assumption 4.4 (Parser soundness). *Every parser $j \in \Pi_{p,v,k_o}^{\text{auth}}$ is sound for L_{p,k_o} in the direction of Equation (2): whenever its report satisfies $\text{CompleteOK}_p(\rho_j(y), k_o, y)$, then $y \in L_{p,k_o}$.*

Assumption 4.5 (Strict semantic construction). *A handler on a strict semantic route serializes its output solely from the decoded intermediate representation and never copies or references a byte range of the source x as an output component.*

Assumption 4.6 (Publication primitive). *Publication uses an exclusive-create primitive with the non-overwrite semantics in Equation (14). On failure, the primitive creates no target and replaces none. If q already exists or is created concurrently, q remains unchanged.*

Theorem 4.7 (Conditional canonical delivery). *Under assumption 4.4, if $\text{Accept}_{p,v}(u, a)$ holds then $y \in L_{p,k_o}$. Under assumption 4.5, a strict semantic route copies or references no source payload as an output component. Under assumption 4.6, a non-clean result creates or replaces no target through the component API.*

Proof. $\text{Accept}_{p,v}(u, a)$ contains the conjunct $\text{Valid}_{p,v,k_o}(y)$ (Equation (10)), which requires $\Pi_{p,v,k_o}^{\text{auth}} \neq \emptyset$ and $\text{CompleteOK}_p(\rho_j(y), k_o, y)$ for every configured j . Taking any such j , assumption 4.4 gives $y \in L_{p,k_o}$. The payload-independence and publication clauses follow directly from assumptions 4.5 and 4.6, respectively, together with Equation (13), which withholds bytes on every non-clean result. $\square$

Theorem 4.7 is deliberately shallow given its assumptions: the acceptance predicate is engineered so that soundness of the hand-written authorizing parsers is the sole load-bearing premise of canonical membership. Assumption 4.4 is therefore the central limitation of the analytical contribution. It is not discharged here; the differential and fuzzing protocol of Section 10 reduces, but cannot prove, the risk of an unsound parser, and a mechanized correctness proof of the registered grammars remains future work. Theorem 4.7 permits coincidental byte equality and does not exclude harmful decoded meaning, and none of its clauses proves codec correctness or malware absence.

5. Format-Specific Reconstruction Methods

5.1. Common reconstruction and validation requirements

Each source-to-output profile mapping instantiates Equation (11) by specifying a constrained output language, minimum reconstruction level, generated name and media type, resource bounds, and an authorizing parser reached through a code path separate from the constructor. This design follows the language-theoretic principle that acceptance should be defined by an explicit language instead of a catalogue of malformed cases [6, 7]. It also addresses the practical risk that two file processors assign different boundaries or meanings to the same bytes [5, 9].

Complete parsing is stronger than recognition of a prefix. For candidate $a = (y, n_o, m_o, k_o, \ell, e)$, the authorizing parser must begin at zero, reach the unique terminator or declared outer boundary, consume $|y|$ octets, and admit every emitted component at its exact parent and position. The output name, declared media type, byte recognizer, and parser report must identify k_o . Resource checks use both y and the recorded reconstruction evidence e . They cannot be inferred from y alone. These conditions establish conformity to the registered output language under the stated assumptions. They do not establish that arbitrary accepted media are harmless.

The contract for a permitted source profile s , reconstruction level ℓ , and output profile k is recorded as

$$\Phi_{p,s,\ell,k} = \left(\Omega_p, q_{p,s}, L_{p,k}, \Gamma_{p,k}, \Pi_{p,v,k}^{\text{auth}}, \mathcal{N}_{p,k}, \mathcal{M}_{p,k}, \mathcal{B}_{p,s,k} \right), \quad (15)$$

where $\Gamma_{p,k}$ is the location-aware component and permitted-metadata policy, $\mathcal{N}_{p,k}$ and $\mathcal{M}_{p,k}$ are the generated-name and output-media-type rules, and $\mathcal{B}_{p,s,k}$ contains input, reconstruction, and output bounds. The remaining symbols were defined in Sections 2 and 4. Reconstruction strength follows

NotProcessed $\prec$ Structural $\prec$ Semantic. The instantiated routes and the limits of their claims are compared in Table 4.

Table 4: Comparison of implemented reconstruction routes. “Retained source” identifies content that survives a structural route and limits its claim.

Input profile	Levels	Retained source	Complete-output evidence	Authorized claim
PNG, JPEG, static WebP	Semantic	Decoded raster only	Grammar parse and second decode	Canonical grammar, exclusion of forbidden source metadata, and source-payload independence
GIF	Semantic	Decoded frames and normalized timing	Block parse, trailer closure, and second decode	Animation closure and source-payload independence within the frame model
APNG	Structural	IDAT/fdAT payloads	Chunk, CRC, sequence, and frame checks	Container closure, exclusion of forbidden source metadata, and bounded animation structure
Animated WebP	Structural	VP8/VP8L frame payloads	RIFF closure, frame checks, and full decode	Container closure, exclusion of forbidden source metadata, and bounded frame consistency
MP3	Structural or semantic	MPEG frames structurally, decoded samples semantically	Exact frame walk or semantic MP3 reparse	Structural tag and suffix removal. Source-payload independence only semantically
WAV PCM	Structural or semantic	PCM bytes structurally, decoded samples semantically	RIFF arithmetic or semantic MP3 reparse	Structural container closure. Source-payload independence only semantically
FLAC	Structural or semantic	Encoded frames structurally, decoded samples semantically	FLAC frame checks or semantic MP3 reparse	Structural stream closure. Source-payload independence only semantically
Ogg Opus/Vorbis	Structural or semantic	Packets structurally, decoded samples semantically	Ogg closure or semantic MP3 reparse	Structural stream closure. Source-payload independence only semantically
ISO BMFF	Structural or semantic	Samples in <code>mdat</code> on the structural path	Box grammar, sample tables, and extent coverage	Container and offset closure. Payload independence only after semantic reconstruction
WebM	Structural or semantic	Blocks structurally, decoded streams semantically	EBML closure or semantic MP4 reparse	Structural container claims. Source-payload independence only semantically

Policy fixes $\Phi_{p,s,\ell,k}$ before dispatch. The handler checks $\mathcal{B}_{p,s,k}$, reconstructs the object, and reports the level performed. The authorizing parser then evaluates the complete candidate against $L_{p,k}$ and $\Gamma_{p,k}$ before rebuilt-output scanning and publication. Overflow, an incomplete boundary, a forbidden component, an exhausted budget, or $\ell < q_{p,s}$ prevents delivery.

The strict policy maps supported audio sources to an MP3 output profile and supported video sources to an MP4 output profile after semantic reconstruction. The source profile and delivered profile therefore need not be identical. The reference encoder, concrete codec settings, and authorizing-parser identifiers are implementation parameters reported in Section 8. A WebM label in the evaluation denotes the source profile. Its semantic route emits an MP4 candidate.

Component closure and metadata policy are related but not identical. Closure defines the complete set of legal structural units and their permitted locations. Metadata policy determines which legal but nonessential units may survive. The evaluated profiles use removal or replacement rather than source-field copying. A byte-substring scan is not used to establish removal, because an arbitrary byte sequence can occur inside compressed data. Absence is established by reparsing the output grammar and observing that no forbidden component is present.

Size and offset integrity are also treated as parser obligations. For a length-delimited component c at offset o_c with header length h_c and declared payload length ℓ_c , the parser requires checked arithmetic and the containment relation

$$o_c + h_c + \ell_c \leq b_{\text{parent}}, \quad [o_c, o_c + h_c + \ell_c) \subseteq B_{\text{parent}}. \quad (16)$$

For sequential grammars, the next component must begin exactly at the previous component’s end. For offset-bearing grammars, a referenced extent must lie in an allowed media-data region and must agree with the relevant count and sample tables. Checks precede slicing, allocation, or offset conversion. These rules turn complete consumption into a constructive invariant rather than an end-of-function comparison alone.

5.2. Static raster images

JPEG, PNG, static WebP, BMP, and TIFF are accepted as static raster input classes when the image library can probe and decode them under policy limits. The output profiles specialize the corresponding JPEG, PNG, and WebP specifications rather than treating successful decoding as the grammar definition [22, 23, 24]. Only PNG, JPEG, and static WebP are output classes. The handler first probes dimensions without decoding the full pixel buffer. Zero dimensions, probe failure, or a probed pixel count above the policy limit block before full decode. After decode, dimensions and pixel count are checked again. If the decoded pixel count exceeds twice the probed count, the operation is rejected. Any other nonzero discrepancy produces a warning and therefore remains non-deliverable under enforcement mode, while both

counts remain subject to their respective policy bounds. The intermediate state is a decoded raster, so the resulting method is semantic reconstruction. BMP and TIFF source syntax, embedded profiles, and application records do not survive because those containers are not copied into the output.

The selected encoder creates a new raster object from decoded pixel state. The implementation’s image representation does not carry source EXIF, XMP, ICC, comment, or thumbnail records through this path. The authorizing parser then performs both structural parsing and a second decode. The second probe and decode must agree on dimensions and remain below the validator’s pixel bound. This post-reconstruction decode matters because structural correctness of marker or chunk boundaries alone does not establish that the generated compressed payload is consumable by the selected decoder.

For PNG, the accepted grammar is

$$\text{Sig} \parallel \text{IHDR} \parallel [\text{PLTE}] \parallel [\text{tRNS}] \parallel \text{IDAT}^+ \parallel \text{IEND}. \quad (17)$$

The parser verifies the PNG signature, every chunk length, every CRC, the single 13-byte IHDR, color-type constraints, palette and transparency compatibility, contiguous nonempty image-data chunks, and an empty terminal IEND. No ancillary chunk other than the profile-permitted tRNS is admitted, and unknown chunks are rejected. Animation chunks are forbidden in the static profile. The terminal offset must equal file length. These constraints implement a narrower language than the full PNG specification [23].

For JPEG, the output begins with SOI and one pinned generated JFIF APP0 segment. Its identifier, version, density unit, nonzero density fields, and zero thumbnail dimensions are checked. The validator uses an explicit marker allowlist and admits baseline SOF0 and progressive SOF2 output. It checks frame dimensions and component table lengths, requires valid scan headers, walks entropy-coded data with JPEG stuffing and restart-marker rules, and ends at one EOI. Other APP segments and COM are forbidden. Marker lengths must remain within the file and no byte may follow EOI. The profile therefore excludes source application segments rather than attempting to distinguish benign from dangerous values inside them.

For static WebP, the validator treats the file as a RIFF form. It requires the outer size to satisfy $\text{RIFFSize} = |y| - 8$, the form type WEBP, exact chunk boundaries, and the specified even-byte padding. Exactly one VP8 or VP8L image payload is present. When VP8X is emitted, its feature flags, dimensions, animation state, and alpha declaration must agree with the present chunks. Animation and metadata chunks, including ANIM, ANMF, EXIF, XMP, and ICCP, are excluded. A second WebP decode supplies the semantic

postcondition.

5.3. GIF, APNG, and animated WebP

GIF uses semantic frame reconstruction under a constrained specialization of the GIF89a block grammar [25]. The handler decodes frames to RGBA, checks the configured frame count and per-frame pixel limit, and accumulates a separate total-pixel budget. Frame delays below two centiseconds are normalized to two centiseconds. The sum of frame delays for one animation cycle is bounded by 60 seconds. This bound does not limit total viewer time when the generated loop declaration requests indefinite repetition. Re-encoding constructs a fresh GIF with generated color tables and that explicit loop declaration. Source comment, plaintext, and application-extension data are not copied. Frame offsets, dimensions, disposal operation, transparency, and normalized delay are represented in generated frame records.

The authorizing parser accepts GIF87a and GIF89a signatures, requires one graphic-control block per frame, and permits only the generated NETSCAPE2.0 loop extension. It checks the logical screen descriptor, color tables, image rectangles, Lempel–Ziv–Welch (LZW) sub-block chains, and one terminal trailer. Unknown extensions and trailing bytes fail validation. A second decoder must read at least one frame and reach end of input. The current parser does not couple extension use to the GIF89a signature, so that version-extension check remains a release condition. The semantic boundary is decoded frame content under the stated delay normalization. Pixel fidelity is evaluated separately.

APNG differs because the implemented handler is a structural chunk remux. It walks the PNG chunk sequence, validates chunk boundaries and CRCs, retains the compressed IDAT and fdAT payloads, and removes ancillary chunks not needed by the constrained animation. The output grammar is

$$\text{Sig} \parallel \text{IHDR} \parallel [\text{PLTE}] \parallel [\text{tRNS}] \parallel \text{acTL} \parallel \text{Frames} \parallel \text{IEND}. \quad (18)$$

There is one **acTL** before image data. The default IDAT image may be the first animation frame when preceded by its **fcTL**, or it may remain outside the animation when the first **fcTL** follows IDAT. Each **fcTL** describes a nonzero rectangle within the canvas and uses defined disposal and blend operations. The shared **fcTL**/**fdAT** sequence starts at zero and is gap-free and duplicate-free. Every controlled frame has data, and the parsed frame count equals the **acTL** declaration. Only IHDR, conditionally required PLTE and tRNS, acTL, fcTL, IDAT, fdAT, and IEND are allowed.

Because `IDAT` and `fdAT` payloads survive, APNG instantiates the structural claim in Table 4. A policy that requires semantic reconstruction blocks this route.

Animated WebP has a separate structural profile rather than being admitted by the static WebP parser. The handler requires an exact RIFF boundary, one 10-byte `VP8X` first with the animation feature set, one `ANIM` before all frames, and at least one `ANMF`. Canvas dimensions, frame rectangles, frame count, per-frame pixels, and aggregate pixels are bounded. Each frame contains either an optional `ALPH` followed by one nonempty `VP8` payload, or one nonempty `VP8L` payload without a separate `ALPH` chunk. The frame alpha state and the `VP8X` alpha flag must agree. Chunk sizes and even-byte padding are consumed exactly. The handler removes `ICCP`, `EXIF`, and `XMP` chunks, clears their `VP8X` feature bits, rejects every other top-level component, and regenerates the RIFF size.

The `webp-animated-canonical-v1` validator, invoked separately from the constructor, repeats the complete structural parse and additionally checks encoded frame dimensions, alpha-state consistency, and agreement between parsed and decoded frame counts. It decodes every emitted frame under fixed frame and aggregate-pixel bounds and verifies the declared canvas. This corroborates decoder consumption but remains structural because the source `VP8` or `VP8L` payloads survive. A policy requiring semantic reconstruction therefore rejects this route.

5.4. *MP3, WAV, and FLAC*

The MP3 structural path recognizes and removes an initial `ID3v2` region and a terminal `ID3v1` tag, then validates the remaining bytes as a nonempty sequence of length-accounted MPEG audio frame extents with admissible headers [26]. The frame parser checks sync, MPEG version, layer, bitrate index, sampling-rate index, and padding. It computes each frame length from the header and requires the next header at exactly that boundary. The canonical output profile admits no `ID3`, `APE`, `Lyrics3`, arbitrary padding, inter-frame material, or suffix. Stream parameters are required to remain stable under the implemented profile. Retained MPEG frames limit this path to the structural MP3 claim in Table 4.

WAV is reconstructed as a new RIFF/WAVE container around retained sample bytes under the RIFF interchange model [27]. The input walk checks RIFF size, chunk boundaries, and padding while locating one supported `fmt` chunk and one `data` chunk. Source ancillary chunks are omitted. The output uses a 16-byte format body for integer PCM, followed by the data chunk and generated zero padding when needed. The validator requires exactly that order and no other chunks. It checks nonzero channel count and sample rate,

supported sample width, block alignment, byte rate, and data divisibility. These conditions require

$$\begin{aligned} \text{blockAlign} &= C \frac{B}{8}, \\ \text{byteRate} &= F_s \text{blockAlign}, \\ |\text{data}| \bmod \text{blockAlign} &= 0, \end{aligned} \tag{19}$$

where C is channel count, B is bits per sample, and F_s is sample rate. The RIFF size includes any generated pad byte and equals $|y| - 8$. Although PCM sample bytes are retained, there is no nested compressed codec bitstream in this constrained profile. The reconstruction is still reported as structural because sample semantics are not decoded and regenerated.

FLAC structural reconstruction follows the native stream organization standardized in RFC 9639 [28]. It retains the 34-byte STREAMINFO payload and encoded audio frames while removing every other metadata block. It emits the **fLaC** marker, exactly one STREAMINFO block marked as the final metadata block, and one or more retained frames. VORBIS_COMMENT, PICTURE, APPLICATION, SEEKTABLE, CUESHEET, PADDING, and unknown metadata blocks are absent from the output grammar.

The separate FLAC authorizing parser is deeper than a metadata-chain walk. It checks STREAMINFO block-size, frame-size, sample-rate, channel, bit-depth, and total-sample fields with checked arithmetic. For each frame it parses the sync and blocking strategy, canonical coded frame or sample number, block-size and sample-rate codes, channel assignment, explicit bit depth, header CRC-8, and frame CRC-16. It traverses constant, verbatim, fixed-predictor, and LPC subframes, including wasted-bit declarations, Rice and escaped residual partitions, and zero alignment. Frame numbering, aggregate sample count, and parameters are cross-checked with STREAMINFO. Streamable-subset limits are applied to block size, low-rate LPC order, and Rice partition order. Every frame end becomes the next frame start, and the final frame ends at EOF.

These checks establish a constrained, frame-CRC-consistent FLAC grammar. Predictor output and STREAMINFO’s MD5 over decoded samples are not recomputed, so retained frames remain outside payload-neutralization claims.

5.5. Ogg Opus and Vorbis

The implemented Ogg method accepts one logical Opus or Vorbis stream under the page model defined by RFC 3533 [29]. The handler validates page capture patterns, version, lacing boundaries, CRC, serial number, se-

quence continuity, and BOS/EOS state. It requires a recognized identification packet, a replaceable comment packet, subsequent media, and a terminal EOS page. Reconstruction replaces the comment packet with a minimal generated OpusTags or Vorbis comment. The current replacement is limited to a comment packet wholly contained in one page and not sharing that page with another packet. Inputs that would require a more general repagination are blocked rather than approximated.

The authorizing parser reconstructs packets across page lacing tables. Continuation flags must agree with whether the prior page ended in an unfinished packet. Exactly one serial number is allowed, page sequence numbers are contiguous, every page CRC is valid, and EOS ends the stream at EOF. Codec header and comment order are checked against the selected Opus or Vorbis grammar. The Vorbis packet obligations follow the codec specification, while Opus follows the Ogg framing and header profile in RFC 7845 [30, 31]. Encoded media packets remain structural evidence only. The authorized claim is stated in Table 4.

5.6. ISO BMFF

MP4, MOV, and M4A inputs enter a common ISO base media file format path. The profile is a constrained use of the ISO BMFF box model and its registered code points [32, 33]. The native handler performs structural rewriting. It removes excluded metadata and free-space structures, recursively rebuilds permitted boxes, normalizes selected timestamps, language fields, handler names, and visual compressor strings, and retains samples in `mdat`. The implementation bounds nesting depth and atoms per level. Because removal changes layout, chunk offsets are adjusted only after the transformed top-level arrangement is known. The current native fix-up uses a checked displacement for collected `stco` and `co64` tables. For audio routing, the media-type aliases `audio/mp4`, `audio/x-m4a`, and `audio/m4a` converge on the same M4A input profile. This alias convergence is necessary for recordings produced by common mobile media APIs, but it does not define a new output grammar. The authorizing parser accepts a narrower nonfragmented, single-`mdat` grammar so that this offset model can be checked end to end.

The canonical hierarchy contains one profile-allowed `ftyp` brand set, one structurally complete `moov`, and one `mdat`. Box sizes may use standard or extended forms only when their declared end lies within the enclosing parent. Unknown and forbidden children fail according to parent location. The validator excludes `udta`, metadata-bearing `meta`, `uuid`, free space, attachments, arbitrary vendor boxes, and bytes outside top-level boxes. The structural rewriter also removes records excluded by the constrained profile, including `sgpd`, `sbgp`, `sdtg`, `stps`, `cslg`, `iods`, `elng`, and `tref`. Some of these boxes

can affect sample grouping, dependency, presentation, seeking, or inter-track relationships. Their removal therefore supports a narrower container-closure claim, not semantic equivalence or general playback compatibility. Structural delivery is confined to registered source profiles for which the policy permits this loss. Otherwise semantic transcoding is required. Compatibility remains an empirical fidelity outcome. Permitted `fiel` and `chrm` sample-entry children are accepted only with their constrained field values and lengths. The validator checks mandatory children of movie, track, media, media-information, and sample-table boxes. Data references must be self-contained. Handler type, sample-entry codec, dimensions, channel fields, timescales, durations, and edit lists are constrained.

Sample-table integrity is established jointly rather than box by box. Time-to-sample counts, composition counts when present, sample-size counts, sync-sample indices, sample-to-chunk runs, and chunk-offset counts must describe one coherent sample sequence. For every chunk j , the validator derives the number and size of its samples and requires

$$[o_j, o_j + L_j) \subseteq B_{\text{mdat}}, \quad (20)$$

where o_j comes from `stco` or `co64` and L_j is derived from `stsc` and `stsz`. Fragment boxes are outside the current output grammar. Across all tracks, the derived extents are sorted and must cover the single `mdat` payload exactly, without a gap or overlap. A semantic transcoding path may produce an MP4 candidate, but its output is normalized by the structural rewriter and must pass the same authorizing parser before publication.

The structural path establishes container closure and offset consistency for the constrained nonfragmented profile. Payload independence requires the semantic path. Strict and paranoid policies block when it is unavailable.

5.7. WebM

The implemented EBML output profile is WebM only. Matroska is not promoted to support by shared EBML recognition. The profile is defined against the EBML and Matroska element model while admitting only the WebM subset named by the output grammar [34, 35, 36, 37]. The structural handler requires one bounded EBML header with `DocType=webm`, one Segment, one Info element, one Tracks element, and at least one Cluster. It removes SeekHead, Cues, Tags, Attachments, Chapters, Void, checksums that become stale, and cluster-relative Position and PrevSize records. Info is rebuilt in a fixed order. Source application names, date, title, and unsupported fields are removed, while MuxingApp and WritingApp are set to a fixed implementation identifier. Segment and child sizes are regenerated

with checked EBML variable-length integers.

The `webm-constrained-v2` validator reparses the full nested grammar on a code path separate from construction. Element identifiers must use their shortest valid VINT encoding. Declared sizes must consume their parents exactly. Nonminimal size widths are accepted because EBML permits them, while unknown size is allowed only for the root Segment whose end is fixed by EOF. The accepted Segment orders Info, Tracks, and Clusters and excludes unknown children.

The track profile permits one VP8, VP9, or AV1 video track and at most one Opus audio track. Track numbers and unique identifiers are distinct. Dimensions, pixel count, duration, timestamp scale, sample rate, and channel count are bounded. The validator checks VP9 feature records, AV1 configuration-record shape and OBU boundaries, and the complete Opus identification header, including mapping, pre-skip-derived CodecDelay, and SeekPreRoll. Agreement between AV1 configuration fields and an optional Sequence Header OBU remains a codec-semantic obligation and is not claimed.

Each Cluster begins with one Timestamp and contains only SimpleBlock media. Block track numbers must identify declared tracks. Reserved flags and all supported lacing modes are parsed to positive frame extents. Video lacing is excluded. Cluster, block, frame, aggregate frame-count, timestamp, and one-hour duration bounds are enforced. Absolute block timestamps are nondecreasing across Cluster boundaries, declared Duration cannot end before the final block timestamp, and each declared track must carry a frame. The last parsed Segment byte must be the last file byte.

The native WebM validator closes the container grammar without decoding VP8, VP9, AV1, or Opus frames. A structural WebM candidate therefore cannot satisfy a policy that requires semantic reconstruction. The strict route decodes the source and emits validated MP4 rather than claiming semantic WebM output.

5.8. *Post-reconstruction validation and deferred evidence*

Every method ends at the predicate in Equation (11). The authorizing report, produced separately from construction, must identify the expected profile and place its terminal offset at $|y|$ before rebuilt-output signature policy is evaluated. Decoder exit, MIME inference, handler return, and signature scanning cannot substitute for complete grammar validation.

The implementation and unit tests identify where these checks are enforced. Empirical claims are limited by the executed protocol in Section 10. Incomplete runs are reported only as finite observations or negative results.

6. Polyglot and Ambiguous-Carrier Handling

6.1. Classification is not acceptance

A file-type label selects work under policy. It is not an authorization decision. Name evidence, caller-supplied media type, and bounded byte recognition can disagree. A separate authorizing parser can then expose a different whole-object interpretation. Polyglot research shows that several file interpretations can be combined and that detector behavior depends on the chosen grammar and boundary [16, 18]. Parser differentials have the same general consequence even when the carrier was not intentionally constructed as a polyglot [5, 19].

Let $R_p(x, n, m)$ be the eligible route set defined in Equation (6). Selection is admissible only when this set is a one-element set:

$$\text{Route}_p(x, n, m) = k \iff R_p(x, n, m) = \{k\}. \quad (21)$$

Compatibility follows the versioned alias relation, not a textual MIME prefix. For example, `audio/wav` and `audio/x-wav` can select one WAV profile. A media suffix paired with an executable grammar is not such an alias.

A complete parser accounts for the whole object presented to that parser, but source and output evidence have different domains. A selected source parser accounts for ranges retained by structural reconstruction. Semantic reconstruction may discard source bytes after bounded decoding. The authorizing parser accounts for the reconstructed candidate y . Its success does not prove that every byte of source x had one interpretation. Context can expose disguise but cannot override incompatible bytes. A handler’s declared output media type is a claim to test, not proof of the candidate format.

6.2. Distinct ambiguous-carrier constructions

The four carrier constructions differ in byte topology and therefore in the observation most likely to falsify acceptance. Their comparison is shown in Figure 3. Every route still passes the full predicate in Equation (11). The rightmost column names the decisive construction-specific control.

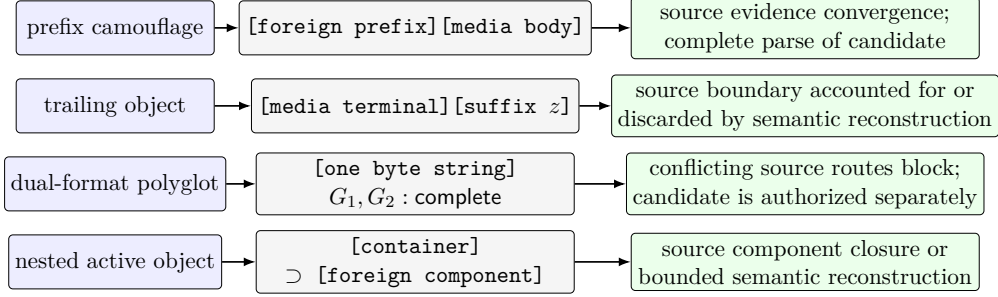


Figure 3: Carrier construction and decisive acceptance control. Prefix camouflage, a suffix, a dual-format interpretation, and a nested component are distinct test strata rather than interchangeable uses of “polyglot.”

Complete consumption directly addresses trailing material in the candidate. For every nonempty suffix z that an authorizing parser leaves after the original terminal point,

$$\text{Parse}_{p,v,k}(y \parallel z) = \rho \wedge \rho.t = |y| < |y \parallel z| \implies \neg \text{CompleteOK}_p(\rho, k, y \parallel z). \quad (22)$$

This implication states non-acceptance by the authorizing parser. It does not infer language nonmembership from parser failure. If the parser instead accepts $y \parallel z$ as a new complete object, all profile and component predicates are evaluated on that entire object. Concatenation alone cannot establish rejection. Declared-size containers apply the same accounting rule to RIFF, ISO BMFF, and EBML parent boundaries.

Component closure rejects a nested foreign object only when it occupies a forbidden structural position. It does not identify hidden meaning inside retained pixels, samples, or codec frames. Prefix camouflage and true polyglots add a source-evidence obligation: detected incompatible routes block. Semantic reconstruction may emit a new object after one selected decoder consumes the source under stated bounds, but acceptance of that output does not establish that the source lacked another unrecognized interpretation. Transformation-time decoder compromise remains governed by Section 7.

6.3. Reconstruction strength under ambiguity

Semantic reconstruction can map several source representations into one output language. A static image carrier is accepted only when the selected decoder produces a bounded raster and the encoder creates a fresh PNG, JPEG, or WebP candidate that passes the authorizing parser and a separate output decoder. GIF follows the same principle at frame level. This supports the claim that source prefixes, trailing objects, excluded container components, and excluded metadata were not copied into the output. It does

not prove that decoded pixels contain no concealed information or that the selected decoder is universally correct.

Structural reconstruction has a narrower effect. APNG can preserve compressed frame payloads. MP3 and FLAC can preserve encoded audio frames. WAV can preserve PCM samples. Ogg can preserve media packets. ISO BMFF can preserve samples in `mdat`, and WebM can preserve block frame payloads. Foreign structure in discarded metadata or beyond an established boundary can be removed or rejected. An alternate interpretation inside retained payload cannot be claimed neutralized. A policy aimed at codec exploitation or payload-level overlap therefore requires semantic reconstruction.

Even when authorizing parsing succeeds, publication still requires $\ell \geq q_{p,s}$ as defined in Equation (8). An APNG or WebM structural candidate can therefore be well formed and remain non-deliverable when policy requires semantic reconstruction. Canonical form cannot silently weaken the requested reconstruction strength.

6.4. Threat terminology and evidentiary boundary

Malware labels are used only to organize scenario provenance, following the behavioral distinctions in Section 2. Method verdicts remain tied to measurable carrier properties such as type disagreement, unsupported grammar, complete-boundary failure, component closure, and reconstruction-level insufficiency. Privacy-bearing metadata belongs to a separate leakage stratum and is not evidence of spyware [13].

Steganographic content and arbitrary data encoded in legitimate pixel or sample values remain outside the structural polyglot claim. Semantic re-encoding can preserve such information because media fidelity is part of the reconstruction objective. Detecting covert meaning requires a separate content-analysis model, corpus, and error specification.

The controlled evaluation therefore treats prefix camouflage, trailing objects, true dual-format polyglots, and nested active objects as separate mutation families. Structural and semantic policies are evaluated independently, with the triggering carrier property recorded for each decision. Until the deferred campaigns run, the justified contribution is the explicit decision rule and the format-specific mechanism by which ambiguity is removed, rejected, or left outside the claim.

7. External Process Supervision and Resource Control

Media reconstruction combines in-process parsers with supervised external programs. Static raster images and GIF are decoded and encoded in-

side the Rust process. APNG, animated WebP, MP4, WebM, MP3, WAV, FLAC, and Ogg structural paths use bounded native parsers that retain selected encoded payloads. Semantic video and audio paths invoke FFprobe and FFmpeg through a process supervisor. These execution modes have different containment properties and support different claims. Structural reconstruction can support container and metadata statements. It cannot support neutralization of a payload retained inside copied pixel, sample, or codec data. A policy requiring semantic reconstruction therefore refuses a structural fallback when a semantic decoder is unavailable.

Every handler dispatch is wrapped by `catch_unwind`. A panic becomes a typed `HandlerPanic` error and candidate bytes are not returned. Panic text is reduced to a bounded printable summary before it reaches logs. This mechanism depends on the checked-in `panic = "unwind"` release profile. A consumer that rebuilds the crate with aborting panics loses this containment property. Unwinding also does not isolate memory corruption in native code, bound allocator growth automatically, or recover from process termination. The in-process image and GIF codecs therefore remain in the trusted computing base for availability and memory safety beyond the guarantees of Rust and their dependencies.

The external-process supervisor creates temporary input and output files, passes local paths to the tools, and disables interactive standard input. FFmpeg receives an explicit protocol allowlist containing only `file` and `pipe`, together with a denylist for network and indirect protocols. Audio reconstruction selects the first audio stream. Video reconstruction selects the first video stream and, when present, the first audio stream. Metadata, chapters, subtitle streams, and data streams are excluded. The generated file remains a candidate. It is read only after a successful child exit and then traverses the same output-size, authorizing-grammar, type-convergence, and signature gates as native output.

The supervisor polls the child every 25 milliseconds. During each cycle it checks whether the monitored output file exceeds the configured byte limit, whether the shared cancellation token is set, and whether the wall-clock budget has expired. Standard output and standard error are drained concurrently to avoid pipe backpressure. Each captured stream is bounded to 16 KiB and later sanitized to remove control characters and bidirectional text controls. A one-second reader-shutdown bound prevents a surviving helper from holding a pipe indefinitely. The `ManagedChild` guard remains armed until normal completion, so an early return or unwinding path still attempts termination and reaping.

On Linux, `SIGXFSZ` is reported as an exceeded output bound rather than an unspecified decoder failure. Across all platforms, the decoder retains

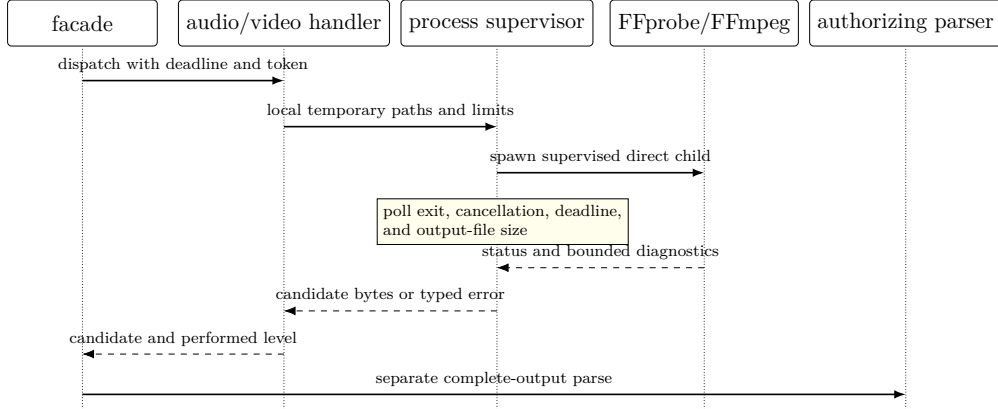


Figure 4: Semantic reconstruction sequence. A successful decoder exit creates only a candidate. Cancellation, timeout, launch failure, oversized output, or failed validation terminates the path without deliverable bytes.

Table 5: Comparison of external-decoder containment by deployment platform.

Platform	Enforced by the component	Guarantee not supplied	Deployment condition
Linux	Process group, no_new_privs , disabled core dumps, CPU/file/address-space limits, deadline, cancellation, group kill, and bounded diagnostics	No namespace, seccomp, chroot, container, or mandatory-access-control boundary	Add a dedicated user or stronger platform sandbox when decoder filesystem access is in scope
macOS	Temporary-file lifetime, deadline polling, cancellation, output-size monitoring, bounded diagnostics, and direct child termination	No Linux process-group, no_new_privs , or resource-limit setup in the compiled branch	Embed in an application sandbox with constrained entitlements and storage access
iOS	In-process checks and fail-closed policy handling	Conventional applications cannot spawn the packaged FFmpeg/FFprobe child path	Strict semantic audio/video blocks unless a separately evaluated in-process backend is supplied

the filesystem authority of its host process unless the enclosing deployment narrows it.

The decoder boundary is a supervision boundary, not a general code-execution sandbox. It bounds direct-child lifetime, diagnostics, and candidate size during ordinary failure or hang behavior on the platforms listed in Table 5. Protocol restrictions constrain media-library input selection. They

do not constrain arbitrary system calls after code execution is compromised. Output authorization and publication gating still apply to a normally returning child. Host integrity, descendant lifetime, and filesystem effects after decoder compromise require an enclosing platform sandbox and are not guaranteed here.

Cancellation is cooperative for in-process work and supervised for external work. `DefenseContext` carries an atomic token, an optional start time, and a millisecond timeout. Handlers call `check_limits` at dispatch and at format-specific loop boundaries. The facade computes a handler budget that does not widen a caller's earlier deadline. `FFmpeg` and `FFprobe` use the remaining duration as an upper bound. A cancellation or timeout is a terminal error, not a warning, and the filesystem publication function is never called on that result. Cooperative checks cannot preempt a single long-running in-process codec call, which is another reason not to describe the in-process path as hard real-time containment.

Temporary files have two roles. Decoder scratch files are named, guard-managed objects created through the `tempfile` crate and removed when their guards drop, subject to host-filesystem behavior. Deliverable publication uses a separate same-directory temporary file. `defend_path` first rejects input and output paths that identify the same entry and refuses a pre-existing destination without modifying it. It runs the full byte pipeline, writes only clean bytes to the temporary file, calls `sync_all`, and invokes `persist_noclobber`. The generated suffix must agree with the validated output profile before staging. The supported claim is non-overwrite: a pre-existing or concurrently created destination is not replaced. Atomic visibility depends on the pinned crate version, operating system, and filesystem primitive and must be verified for the deployment. The parent directory is not synchronized, so crash durability after power loss is not claimed.

At the result boundary, a blocked candidate may be summarized in an in-memory quarantine envelope containing its reason, digest, sizes, and stage details. The envelope is not deliverable, and the current implementation does not persist quarantined content. Every non-clean verdict returns an empty byte vector, zero delivered size, and the digest of the empty string. Every exposed interface enforces the same rule. Parser failure, panic, child launch failure, timeout, cancellation, policy ambiguity, and authorizing-validation failure therefore share one no-publication postcondition.

8. Reference Implementation

The AURA FILE DEFENDER reference implementation targets Rust 2024. Its production crate contains dispatch, format handlers, validation, and publi-

cation, while two workspace crates retain compatibility and interface schemas. The reviewed snapshot is commit 823a5ec7a0dd. The retained evidence records the full digest. Optional policy authentication uses a hash-based message authentication code (HMAC). Figure 5 shows the separation of policy construction, media processing, and publication.

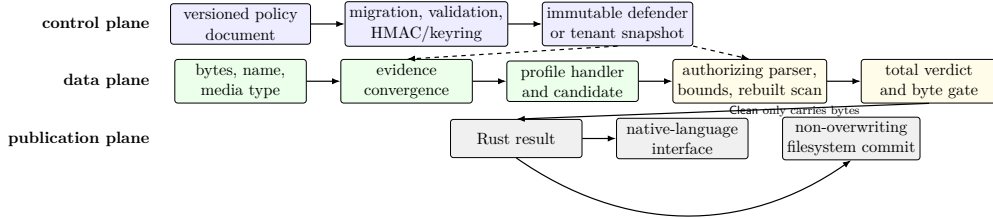


Figure 5: Reference implementation planes. Policy state is validated and frozen before request processing. HMAC authentication is optional and configured separately. Handlers produce candidates, whereas the validation and verdict stages alone authorize bytes for an interface or final path.

Construction of **FileDefender** normalizes media allowlists and denylists into lookup sets and compiles baseline plus administrator signatures into one Aho–Corasick automaton. Request processing uses this immutable state. The hot path performs no policy read or network lookup.

The first stage checks cancellation, input length, portable-name rules, blocked terminal extensions, and dangerous suffix chains. It derives profiles from the name, supplied type, and bytes. Strict policy treats incompatible, missing, or unknown evidence as a typed error. Precedence cannot override disagreement because convergence is checked before dispatch and again on the rebuilt type.

An optional signature pass can reject before decoding. It is an early policy check, not an acceptance proof. The mandatory post-reconstruction pass scans the candidate with location-aware rules. Baseline patterns include the EICAR test string and leading executable, bytecode, script, and archive markers. Administrator indicators are compiled into the immutable engine. Exact byte matching detects configured indicators but cannot infer behavior or malware family.

Handlers dispatch by media kind, while agreement is recorded at the more specific format-profile level. The implemented routes are compared in Table 4. Semantic audio uses FFmpeg with `libmp3lame` and emits `audio/mpeg`. Semantic video uses H.264 with `yuv420p`, optional AAC audio, and the `faststart` MP4 layout. Both paths remove source metadata and chapters before the candidate enters the authorizing parser. Unsupported or non-media input reaches a generic handler that reports an unprocessed level and

Table 6: Implementation traceability at reviewed source revision 823a5ec7a0dd. The table records source boundaries, not empirical results.

Obligation	Primary implementation	Enforced evidence or terminal effect
Input convergence	<code>src/types.rs</code> , <code>src/lib.rs</code>	Name, supplied type, and byte profile are compared before dispatch
Policy strength	<code>src/policy.rs</code> , <code>src/lib.rs</code>	Per-kind required level is compared with the handler-reported level
Bounded reconstruction	<code>src/handlers/*.rs</code> , <code>src/process.rs</code>	Dimensions, counts, durations, process budgets, and cancellation yield typed failure
Component closure	<code>src/validation.rs</code>	A separate parser reports profile, component count, and complete bytes consumed
Signature policy	<code>crates/defender-signatures</code> , <code>src/lib.rs</code>	Pre-scan is configurable and rebuilt-output scanning precedes verdict derivation
Delivery after a clean verdict	<code>src/lib.rs</code> , <code>src/ffi.rs</code>	Non-clean outcomes expose zero output bytes through Rust and C
Non-overwriting path publication	<code>src/lib.rs</code>	Suffix convergence, same-directory staging, file synchronization, and non-overwriting persistence occur only for clean results. Directory crash durability is not claimed
Tenant snapshot selection	<code>src/tenant.rs</code>	Versioned copy-on-write snapshots resolve to immutable defenders

a blocking alert, never a pass-through path.

Validation separate from construction distinguishes transformation from acceptance. The implementation selects one registered complete-file grammar for the output profile. Its parsers cover PNG, APNG, JPEG, static and animated WebP, GIF, MP3, WAV, Ogg/Opus, FLAC, ISO base media, and constrained WebM. They check terminal boundaries, component placement, required or duplicate structures, specified integrity fields, arithmetic bounds, and internal references. Raster and GIF output also undergo a decode check. A handler declaration is never reused as format evidence.

The facade then compares the declared output media type with a per-kind allowlist, applies a global denylist, and independently sniffs the output bytes. Executable magic at offset zero is always blocking even if output-kind

matching is relaxed. Candidate length is checked against an absolute output bound and, for nonempty input, against a maximum expansion ratio. Failure of any check adds a blocking alert or returns a typed terminal error. The final verdict is **Blocked** when any blocking alert exists, **Suspicious** when no blocker but at least one warning exists, and **Clean** otherwise. Informational alerts record normal transformations without changing the verdict.

Dry-run mode converts blockers into diagnostic warnings and may therefore return **Suspicious**. The controlled profile uses enforcement mode. Dry-run outcomes do not enter accepted counts or carry deliverable bytes.

Policy lifecycle begins with a versioned document. Migration validates finite positive limits, resolves the named base profile, normalizes extension changes, and constructs immutable processing state. Strict and paranoid profiles require supplied media evidence, reject unknown types and decoder failure, require semantic audio and video reconstruction, and prohibit source retention as a fallback. The default profile admits broader diagnostic and structural behavior. Results under different profiles are not interchangeable.

Multi-tenant deployment uses versioned immutable snapshots. Readers resolve one tenant policy without a read lock, while writers construct a replacement snapshot and publish it conditionally. Monotonic version checks reject stale updates and removals. A removed tenant uses the registry fallback policy. Import and export records may bind actor, source, reason, and timestamp provenance.

Snapshot authenticity must be configured explicitly. Plain SHA-256 snapshot objects support checksum comparison but their `verify` method rejects them as unauthenticated. HMAC with SHA-256 envelopes bind a canonical serialized policy payload to a key identifier. Keyring imports reject unknown, expired, or invalid keys and record per-key use counters. Deprecation plus a grace period supports rotation. These APIs assume secure key provisioning and a trustworthy control-plane clock. They do not provide public-key signatures, rollback-resistant durable storage, or confidentiality for policy contents.

The optional native interface validates serialized lengths and text fields before dispatch and converts implementation failures into typed results. It exposes no output for a non-clean verdict, preserving F8 at the interface boundary. Allocation and platform-packaging details belong to the implementation documentation and do not strengthen the process-supervision claim in Section 7.

Decoder failure remains non-deliverable. A policy may retain a bounded, control-sanitized diagnostic in the blocked stage, but that evidence cannot authorize candidate bytes.

Results retain structured alerts, ordered stages, reconstruction strength,

profile, component and consumption counts, sizes, removal counters, and duration. Bounded logs omit input bytes. The current audit sink is inert, so durable delivery, redaction, retention, and correlation remain responsibilities of the embedding application. The reference crate does not persist audit events.

Source traceability identifies each enforcement site. Empirical support remains subject to the parser, fuzzing, external-validation, fault, and performance protocol in Section 10. Section 11 reports the retained bounded run, not a completed controlled experiment.

9. Corpus Design and Ground Truth

Following Sections 2 and 6, the corpus stratifies observable carrier properties rather than malware families. Behavioral and operational labels enter only as independently supported provenance for executable prefixes, disguise, trailing objects, true polyglots, malformed structures, or decoder regressions [13]. Passive metadata is annotated as a privacy-leakage property.

9.1. Unit of analysis and manifest discipline

The unit of analysis is one manifest record under one frozen policy. Each record binds its identifier, relative path, SHA-256 digest, byte length, provenance, license, portable name, supplied type, expected action and reason family, carrier class, format profiles, and mutation recipe. Path escape, duplicate or aliased identities, and digest or size disagreement invalidate the run before processing. Raw malicious or exploit-bearing inputs remain in the isolated corpus and are cited by digest and provenance record.

Ground truth may come from a reproducible generator, complete reference-parser reports, or two-reviewer adjudication. Parser evidence records tool version, configuration, offsets, and profile. The schema checks reviewer or evidence references but not every metadata field, so controlled eligibility also requires review-packet audit. The bounded experiment did not meet that condition. Suffix labels, uploader descriptions, antivirus names, and one media-type guess are insufficient. Disagreement is resolved before freezing or the record is excluded.

Each record has one expected action and reason family. The action oracle compares deliver or block, while the reason oracle compares normalized alert-family prefixes and checks pipeline state separately. Prefix agreement does not prove the first causal stage. An internal I/O failure or caught panic fails the run even when no bytes are published.

The frozen fields, ground-truth sources, denominators, and gates form the evidence contract. A post-outcome change requires a new manifest and run.

9.2. *Stratified corpus*

Each record contains one explicitly frozen primary stratum from Table 7 for aggregate denominators. Secondary tags retain overlapping properties for stratified sensitivity summaries. The primary assignment is reviewed before execution and is not inferred from the order of words in the generation or acquisition record.

S0 and S1 estimate benign false blocking. S2 mutates one obligation before selected interactions. S3 holds bytes fixed while crossing names and types. S4 reserves true polyglot for one octet string completely accepted by two incompatible parsers and records other ambiguous carriers separately. S5 tests $L-1$, L , and $L+1$ where representable. S6 expects blocking. S7 contains minimized, versioned regressions and does not represent a vulnerability class.

9.3. *Safe carrier scenarios*

Synthetic scenarios preserve the carrier property without operational malware behavior. Executable-shaped regions contain inert magic, length, and padding but no instructions, commands, network locations, credentials, or shellcode. Nested-object cases use a structurally recognizable sentinel. Metadata cases use synthetic location, author, comment, thumbnail, or application fields. Decoder regressions must be minimized, legally distributable, and confined to the isolated procedure. The versioned research package retains the full scenario catalogue.

Safe synthesis is suitable for type, boundary, metadata, and resource claims. It is not exploit evidence when a claim depends on decoder control flow. Such cases belong to S7. No synthetic sample is described as live malware.

9.4. *Mutation matrix and pairing*

The matrix in Table 8 lists format-aware mutation families. The protocol records each operator, parsed field or offset, value, interaction, and seed. Generators reparse before mutation and confirm the postcondition. Coverage of a table row does not imply that every listed operator ran.

Pairwise interactions cross boundary, size, metadata, classification, limit, and cancellation conditions. Higher orders require a named mechanism or prior regression. Base and mutation remain paired even when outcomes match.

Table 7: Prespecified corpus strata and their primary inferential roles.

ID	Population	Required coverage	Primary outcome
S0	Canonical benign media	Output profiles selected in the frozen study, boundary sizes, color and channel models, frame and duration classes	False blocking and usability
S1	Benign noncanonical media	Excluded metadata, optional elements, unusual legal orderings, padding, alternate brands, comments, and vendor extensions	Reconstruction coverage and false blocking
S2	Malformed or truncated structures	Lengths, offsets, counts, checksums, sequences, terminators, duplicate mandatory elements, overlap, and budget exhaustion	Fail-closed rejection
S3	Classifier disagreement	Byte format crossed with names, suffix chains, media types, aliases, missing evidence, and malformed display names	Type-convergence enforcement
S4	Multi-grammar and boundary carriers	True polyglots, prefix camouflage, appended or trailing objects, and nested active-object constructions	Whole-string ambiguity, complete consumption, and canonical closure
S5	Resource exhaustion	Size, expansion, dimensions, aggregate pixels, frames, duration, bitrate, channels, nesting, element counts, and lacing	Boundary enforcement
S6	Unsupported or active classes	Executables, scripts, archives, active documents, PDF, vector markup, shortcuts, packages, and unknown bytes	Media-only policy enforcement
S7	Parser and decoder regressions	Legally distributable minimized panic, arithmetic, differential, and decoder cases	Regression resistance

9.5. Sampling, partitions, and denominator integrity

A base artifact is the original acquired or generated object from which one or more mutation records descend. Partitions keep every descendant of the same base artifact together. The current schema records source digests and base relationships but does not enforce decoded-semantic fingerprints

Table 8: Format-aware mutation families from which a frozen protocol selects and records concrete operators and interactions.

Family	Mandatory fields and structures
PNG and APNG	CRC, chunk length, type and order, duplicate IHDR, IEND, or acTL, animation sequence, frame bounds, and suffix after IEND
JPEG	Segment length, nested SOI, premature EOI, APP or COM insertion, entropy-scan truncation, and suffix after EOI
Static and animated WebP	RIFF size and form, chunk size and padding, VP8X flags, duplicate payload, ANIM/ANMF order, frame rectangles and payload dimensions, alpha-state agreement, metadata insertion, and trailing bytes
GIF	Dimensions, color tables, sub-block length, extension type, frame rectangle, delay, missing trailer, and suffix
MP3	Sync, version, layer, bitrate, sample rate, padding, frame length, ID3 size, inter-frame bytes, and trailing bytes
WAV	RIFF size, chunk size and padding, chunk order, duplicate format or data chunks, format arithmetic, and suffix
FLAC	Metadata last flag, type and length, STREAMINFO position, duplication and ranges, blocking strategy, sample-rate and bit-depth codes, frame header, subframe, residual, CRC-8, CRC-16, and suffix
Ogg and Vorbis [29, 30]	Capture pattern, version, lacing, page CRC, serial, sequence, flags, comment continuation, EOS, and suffix
ISO BMFF	Standard, extended, and zero sizes, parent location, duplicate mandatory boxes, offsets, sample and fragment tables, M4A media-type aliases, sample-group, dependency, composition, language, and track-reference boxes, and suffix
EBML and WebM [34, 35, 36]	VINT identifier and size, noncanonical encoding, unknown size, DocType, stale positions, application and date fields, codec configuration, duplicate tracks, undeclared block track, timestamp order, duration coverage, lacing, frame bounds, and budget exhaustion

or automatic deduplication across identifiers. Record-level summaries are therefore record-weighted, and unique source and output digests are reported separately.

Let $B_{\text{ben}} = \{i \in S0 \cup S1 : i \text{ is expected to be accepted}\}$ be the benign acceptance set, and let D contain every record whose expected action is blocking. Expected accepts outside $S0$ and $S1$ contribute to action agreement but not to benign false blocking. Let A_i indicate observed delivery of a clean output for record i . The false-blocking rate is

$$\widehat{\text{FBR}} = \frac{\sum_{i \in B_{\text{ben}}} (1 - A_i)}{|B_{\text{ben}}|}, \quad (23)$$

and the false-acceptance rate is

$$\widehat{\text{FAR}} = \frac{\sum_{i \in D} A_i}{|D|}. \quad (24)$$

Both are record-weighted estimands. They are reported overall and by stratum, concrete format, carrier mechanism, and reconstruction requirement when the subgroup was prespecified. A base-artifact-weighted sensitivity summary is required when mutation multiplicities differ. An expected block that ends in a pipeline error remains a non-delivery for Equation (24), but it also increments the separate pipeline-error gate and cannot be credited as a successful expected rejection. A benign sample blocked because a required external tool is missing contributes to Equation (23). Missing tools are not an exclusion criterion.

Reason-family agreement is reported separately because action agreement can conceal a different blocking reason. Since prefix matching alone does not establish the causal stage, stage agreement is a separate endpoint. A post-freeze ground-truth correction creates a versioned manifest and new run identifier.

10. Reproducible Evaluation Protocol

The prespecified evaluation tests canonical-profile compliance, required reconstruction strength, and fail-closed handling of unsupported, ambiguous, malformed, over-budget, and incomplete artifacts.

10.1. Frozen inputs and execution order

A run identifier binds source state, dependencies, target features, policy and registries, corpus and format digests, external codecs, and server environment. Any change creates a new append-only record. Failed runs are

retained, reruns cite their predecessors, and pooling requires a prior comparison protocol.

The server follows the gated sequence in Figure 6. Commands, environment, status, times, and digests enter the run record. Schema-invalid, incomplete, or hash-inconsistent reports fail their phase.

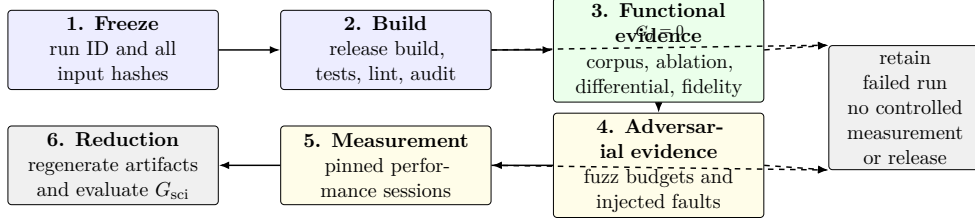


Figure 6: Controlled evaluation sequence. Functional or adversarial failure retains reportable evidence but prevents entry into controlled performance measurement and release-candidate eligibility.

For binary phase-entry and gate variables $E_j, G_j \in \{0, 1\}$, the stopping rule is

$$E_1 = 1, \quad E_{j+1} = E_j G_j, \quad j = 1, \dots, 5. \quad (25)$$

Thus no later measurement compensates for an earlier failed gate. The run record identifies hardware, operating system, toolchains, CPU affinity, observed governor and turbo state, background load, and external-tool identities. A controlled performance run must additionally pin the controls that the platform permits. Missing required evidence makes G_{sci} false.

10.2. Property suite

The eight invariants in Table 3 map one to one to the distinct tests in Table 9. Because the global gate is a conjunction, aggregate success cannot hide one invalid accepted output or one fault case that publishes bytes.

Table 9: Property suite, test oracle, and admissible evidence.

ID	Formal invariant and empirical oracle	Oracle	Evidence unit
F1	Complete consumption	Accepted output parses from offset zero and consumed octets equal output length. An unconsumed suffix prevents acceptance	Accepted output and suffix-mutated pair

Continued on next page

ID	Formal invariant and empirical oracle	Oracle	Evidence unit
F2	Component closure	Every parsed component is permitted at its exact grammar location. Unknown or forbidden components are removed through valid reconstruction or blocked	Parsed component path
F3	Output evidence convergence	Generated name, declared output type, byte recognition, and authorizing report resolve to one output profile	Accepted candidate record
F4	Semantic source-independence	A strict semantic path serializes output from decoded media rather than retained source payload	Source-to-candidate provenance trace
F5	Second-pass stability	Under the same policy and profile, a second pass is exact or satisfies the registered semantic closure	First- and second-pass pair
F6	Resource boundedness	Each declared counter, cancellation condition, and deadline returns a typed failure when its bound is exceeded	Boundary and injected-failure case
F7	Validation separate from construction	Every clean candidate is reparsed by a registered authorizing parser distinct from the constructing handler	Accepted candidate and internal authorizing report
F8	Fail-closed result construction and publication	Every failed conjunct, non-clean verdict, error, race, and persistence failure exposes no deliverable candidate and preserves a pre-existing target	Interface and filesystem trace

F1 appends zero, foreign-object, and malformed suffixes after the terminal extent. F2 checks parsed component paths, F3 compares normalized profiles, and F4 verifies that strict semantic output is serialized from decoded media. Unlike F1–F3, F4 is established by the construction path rather than by an artifact-observable predicate: byte-level source independence across a decode–re-encode step cannot be read off the candidate y alone, so F4 rests on assumption 4.5 and the code path that serializes only from the decoded intermediate representation.

Second-pass stability operationalizes F5 but is not itself an acceptance condition. Let $\text{Out}_{p,v}(u)$ be defined as y only when the pipeline returns a clean accepted candidate $a = (y, n_o, m_o, k_o, \ell, e)$. Let $u[a] = ((y, n_o, m_o), (p, d'))$ be a second request under the same policy and a fresh, equivalent deadline

context d' . For profiles registered as exact,

$$\text{Out}_{p,v}(u) = y \implies \text{Out}_{p,v}(u[a]) = y. \quad (26)$$

For lossy profiles, registered semantic closure instead requires a clean second verdict, the same output profile and reconstruction requirement, and a pre-specified fidelity comparator. It does not imply perceptual equivalence or bounded accumulated drift. The policy comparison in Equation (12) holds the candidate and parser reports fixed. End-to-end comparisons remain descriptive when route selection or reconstruction changes.

10.3. Paired mechanism ablation

The paired study measures decision sensitivity within the frozen corpus, not antivirus accuracy or a deployment-population effect. Let P_0 be the full strict policy and $\mathcal{C}$ the conditions in Table 10. Before ablation, every input is replayed under P_0 . Any mismatch in action, verdict, reason, output digest, reconstruction, or profile aborts the study.

Condition	Single registered change from P_0	Applicable records	Interpretive endpoint
without-input-signature-prescan	Replace input-and-output signature scanning with output-only scanning	All records	Observed paired transition after removing pre-reconstruction signature evidence
without-supplied-type-evidence	For records with a supplied type, omit that value and disable its prerequisite. For records without one, change only the prerequisite	Two prespecified groups analyzed separately. No pooled intervention estimate	Observed paired transition within each distinct intervention group
lowered-reconstruction-requirement-where-configurable	Lower the animated-image, video, and audio requirement from required semantic reconstruction to structural reconstruction	Manifest profiles eligible for one of these configurable requirements	Decision transition under a lower registered reconstruction floor

Table 10: Prespecified paired mechanism-ablation conditions. Condition identifiers match the machine-readable protocol.

Let $\mathcal{I}$ be the full corpus and $I_c \subseteq \mathcal{I}$ the preregistered applicable set for condition c . The evidence stream retains one auditable row for every pair in $\mathcal{C} \times \mathcal{I}$, including an explicit inapplicable row when $i \notin I_c$. Estimands use only I_c . $E_i^{(c)} \in \{0, 1\}$ indicates completion without a pipeline error, and $Y_i^{(c)} \in \{0, 1\}$ is the observed decision for record i , where one denotes acceptance and

zero denotes blocking. The superscript zero denotes the eligible full-policy record. The paired transition counts and full-policy denominators are

$$\begin{aligned} N_{ab}^{(c)} &= \sum_{i \in I_c} \mathbb{1}[E_i^{(c)} = 1 \wedge Y_i^{(0)} = a \wedge Y_i^{(c)} = b], \\ B_a^{(c)} &= \sum_{i \in I_c} \mathbb{1}[Y_i^{(0)} = a], \quad a, b \in \{0, 1\}. \end{aligned} \quad (27)$$

The two directional summaries are

$$q_{1 \rightarrow 0}^{(c)} = \frac{N_{10}^{(c)}}{B_1^{(c)}}, \quad q_{0 \rightarrow 1}^{(c)} = \frac{N_{01}^{(c)}}{B_0^{(c)}}. \quad (28)$$

These are the proportions of full-policy accepts becoming blocks and blocks becoming accepts. Empty denominators are not estimable. Pipeline errors remain outside the 2×2 matrix and fail the execution gate.

Let m_{ci} be the multiplicity of pair (c, i) in the emitted stream. For condition c , let H_c contain its prespecified intervention groups: one group for signature removal, supplied-type-present and supplied-type-absent groups for type ablation, and animated-image, audio, and video groups for the reconstruction-floor ablation. Let $n_{c,h}$ be the applicable count in group $h \in H_c$, and let N_{err} be the pipeline-error count. The predicates G_{strict} , G_{context} , G_{replay} , and $G_{\Delta P}$ verify, respectively, the strict starting settings, preservation of the frozen input and context except for the registered evidence removal, equality of the full-policy replay, and isolation of the condition-specific policy change. Ablation completeness is

$$\begin{aligned} G_{\text{abl}} &= G_{\text{strict}} \wedge G_{\text{context}} \wedge G_{\text{replay}} \wedge G_{\Delta P} \\ &\wedge \bigwedge_{(c,i) \in \mathcal{C} \times \mathcal{I}} [m_{ci} = 1] \wedge \bigwedge_{c \in \mathcal{C}} \bigwedge_{h \in H_c} [n_{c,h} \geq 1] \wedge [N_{\text{err}} = 0]. \end{aligned} \quad (29)$$

The exact-pair term requires a complete $3|\mathcal{I}|$ audit matrix without duplicates or omissions. Inapplicable rows do not enter Equation (27). The proportions are descriptive frozen-corpus properties without a population interval, significance test, causal interpretation, malware-detection rate, or prevalence claim.

10.4. Differential output validation

Every accepted output is checked by the authorizing parser and at least two applicable external format or profile validators. Distinct families use different executable or library lineages. Separate process boundaries alone do not establish diversity. Reports bind output digests to executable identities,

detected profiles, and outcomes. Any missing, rejecting, or disagreeing result fails the gate. These external checks corroborate decodability or profile recognition only. They do not independently establish complete consumption, component closure, the expected action, or the fixture-derived decision oracle, and they cannot exclude common-mode errors.

10.5. Coverage-guided fuzzing and bounded checks

Fuzzing records fixed budgets, seeds, repetitions, sanitizers, corpora, and failure handling, following established evaluation guidance [38]. Targets cover top-level dispatch, complete output validation, and native animated-image, MP4, WebM, Ogg, and audio paths. Structured seeds preserve outer headers while mutation reaches lengths, offsets, checksums, lacing, and sequences. Unique failures are minimized, hashed, reproduced, and added to S7 when appropriate.

Budget exhaustion with no discovered failure means only “no failure observed” under that campaign. Native-interface and policy-deserialization properties receive separate regression and static review, not dedicated fuzz-coverage claims. Bounded checks cover named finite arithmetic and parser domains. Neither fuzzing nor static review verifies an external codec.

10.6. Crash, cancellation, and publication campaigns

The fault schedule exercises cancellation, deadlines, child launch and time-out, bounded diagnostics, output-size termination, validation failure, caught panic, and attempted non-clean publication. The executed cases record the injection point, terminal state, paths, child status, and cleanup against

$$\text{FailureBeforePublication} \implies \text{DeliverableBytes} = 0. \quad (30)$$

The destination is monitored and any pre-existing object is hashed. Allocator, temporary-file, and persistence failures require explicit injection evidence before G_{fault} can pass. Code review can motivate the expected behavior but cannot substitute for an executed fault case.

10.7. Performance protocol

After functional gates pass, ten sessions invoke the public byte-defense API over corpus permutations generated from a recorded seed for each session. The matrix stratifies format, reconstruction level, input and decoded size, media structure, carrier, and outcome. Cold and warm records retain order, time, profile, reconstruction, outcome, and errors. Blocked and timed-out calls remain in their path denominator. Here, “cold” denotes the first timed

invocation after an already constructed defender. Policy normalization and signature-engine construction are outside the timed interval.

Latency summaries report the median, 95th percentile (p95), and 99th percentile (p99). Sequential single-worker service rate is not concurrent throughput. Hashed resource logs provide peak resident set size (RSS). Output expansion for clean deliverable i is

$$e_i = \frac{|y_i|}{|x_i|}, \quad (31)$$

with zero-length input retained as a separate invalid or blocked group.

10.8. Fidelity and usability

Security acceptance and fidelity remain separate. Every accepted output is evaluated under a frozen fidelity policy. Lossless profiles compare the pre-specified decoded dimensions, channels, frames, samples, and timing fields. Structural profiles test the exact properties named by their profile. Lossy paths use prespecified metrics and drift bounds. Tool error, missing or non-finite metrics, threshold failure, or an unevaluated output fails the fidelity gate. Passing fidelity never enlarges the security claim.

10.9. Statistical analysis

Corpus endpoints report x/n under the denominators in Section 9.5. Because purposive mutations are clustered by base artifact, no binomial population interval is attached to decision, differential, fidelity, or invariant outcomes. Zero denominators are not estimable. The first release has no confirmatory policy or version hypothesis. Its paired discordances are descriptive.

Median latency uses 10,000 hierarchical bootstrap replicates over sessions, dependence clusters, and base artifacts [39]. The resulting percentile 95% interval quantifies variation within the controlled design, not deployment-population uncertainty. Expansion is reported as an arithmetic mean without a bootstrap interval. Cold and warm paths remain separate. Exploratory subgroups are labeled, missing observations are not imputed, and every output table traces to its frozen source digests.

10.10. Release gates and claim authorization

Controlled-run and release-candidate eligibility require the conjunction

$$\begin{aligned} G_{\text{sci}} = & G_{\text{prov}} \wedge G_{\text{schema}} \wedge \bigwedge_{r \in \text{corpus}} G_r \wedge \bigwedge_{j=1}^8 G_{\text{F}j} \\ & \wedge G_{\text{abl}} \wedge G_{\text{diff}} \wedge G_{\text{fidelity}} \wedge G_{\text{fuzz}} \wedge G_{\text{fault}} \wedge G_{\text{perf}} \wedge G_{\text{regen}}. \end{aligned} \quad (32)$$

G_{prov} requires frozen source, dependency, policy, corpus, tool, and environment identities. G_{schema} requires every report to pass its versioned schema and cross-file consistency checks. Each G_r requires the expected action, reason family, pipeline state, digest, profile, and per-record evidence for manifest record r . The predicates $G_{F1}, \dots, G_{F8}$ correspond to the eight rows of Table 9. G_{diff} and G_{fidelity} require complete external-validation and fidelity records for every accepted evaluation. G_{fuzz} , G_{fault} , and G_{perf} require their full prespecified budgets and cases. G_{regen} requires every table and summary to regenerate from the retained raw records. Together with the ablation gate, these predicates define controlled-run eligibility. A failed conjunct precludes controlled, population, capacity, or release-candidate claims but does not suppress finite observations or negative results. Failed runs remain append-only, and a release candidate additionally requires `release_eligibility.eligible=true`.

11. Bounded Server Experiment

The experimental path was exercised under a two-CPU, 12-GiB Linux control group (cgroup) on a shared server with strict policy v1 (SHA-256 prefix `0e5ede36365b`). The evidence package retains the full digest. This bounded descriptive experiment did not replace the controlled release run in Section 10. Its estimands were finite-corpus decision agreement, authorizing-parser acceptance, external format or profile agreement, fidelity, fault and fuzz outcomes, and warm latency. Table 11 reports selected retained outcomes and their evidentiary limits.

Table 11: Selected retained outcomes for the bounded server experiment. Counts and proportions describe only the generated corpus and executed schedules.

Evidence area	Observed result	Evidence boundary
Provenance and validation	Run <code>20260720T130631Z</code> . Corpus, fidelity, performance, and fuzz evidence originated at <code>632713401c57</code> . external-validator and fault rechecks at <code>b1b193238dfe</code> . Runner hardening occurred at <code>80c2073b6aed</code> , and packaging at <code>823a5ec7a0dd</code>	The campaign was not rerun under the final hardened runner. All 241 Rust and 118 research tests passed. One fixture generator was ignored. Format, Clippy, package, native-toolchain, 3 benchmark-smoke, and 7 fuzz-target checks passed
Corpus decisions	104 generated records: $S0 = 50$, $S1 = 30$, and inert $S6 = 24$. All 80 expected-clean evaluations delivered and all 24 expected-block evaluations were blocked	0/80 expected-clean records were withheld and 0/24 inert unsupported records were delivered. No action, reason-family, or pipeline-state mismatch was recorded

Continued on next page

Evidence area	Observed result	Evidence boundary
Closure and external validation	32 exact and 48 registered semantic second-pass closures. 80/80 record-level external checks passed for 40 unique output digests	Each accepted record used at least two applicable tools drawn from ImageMagick 6.9.12-98, FFprobe/libavformat 6.1.1-3ubuntu5, and MediaInfo 24.01. These checks established profile recognition or decodability, not canonical closure
Fidelity	24/24 static, 8/8 GIF, 0/32 audio, 8/8 MP4, 0/8 source-WebM. 40/80 overall	Audio duration drift 6.16–12.95% with per-record mean scale-invariant signal-to-distortion ratio (SI-SDR) 40.09–59.34 dB. Source-WebM structural similarity index (SSIM) 0.667352–0.712112 with drift $\leq 0.1604\%$. No failure was waived
Fault and fuzz campaigns	11/11 selected single-test fault checks passed. Seven fuzz targets each received one seeded run with a requested 30-second budget, for 210 requested target-seconds	7,299,439 iterations produced no recorded crash, sanitizer finding, timeout, or crash artifact. The protocol requested three one-hour repetitions per target
Performance	1,872 observations = 312 cold + 1,560 warm. Cold median/p95 1.934/382.067 ms. Warm 1.906/358.892 ms. Peak RSS 64,308 KiB	Warm observations equal 104 records $\times$ 5 repetitions $\times$ 3 sessions and are not independent samples. Warm mean 125.936 ms/call corresponds to 7.94 sequential calls/s for this order and host. No bootstrap interval was produced. Warm median/p95: unsupported 0.016/0.040, raster 0.625–0.860/0.998–1.284, GIF 1.906/3.561, audio 228.799–228.975/279.765–304.897, video 356.371–357.113/432.328–433.799 ms
Mechanism ablation	Three conditions were prespecified but not executed in this bounded package	No paired transition or mechanism-contribution claim is authorized
G_{sci} and release candidate	Ineligible	S2–S5 and S7 absent. Required public-benign coverage absent. Shortened fuzz and performance schedules. Thirty-three other containers active on the host. External tools were mutable. No append-only publication. Fidelity gate failed

Fidelity used the versioned policy `aura-fidelity-policy-v2`. Its SHA-256 digest begins with 703526016497. Static images required equal dimensions and peak signal-to-noise ratio (PSNR) ≥ 40 dB. Animated images additionally required equal canvas and frame count, timing drift $\leq 2\%$, and frame PSNR ≥ 40 dB. Video required equal dimensions, duration drift $\leq 2\%$, and SSIM ≥ 0.90 . Audio required equal channels, duration drift $\leq 2\%$, and SI-SDR ≥ 20 dB. Audio failed only the duration threshold. Source-WebM inputs, semantically emitted as MP4, failed only SSIM.

The full policy matched the fixture-derived expected action, reason family, and pipeline state for all 104 records. The 80 accepted record-level evalua-

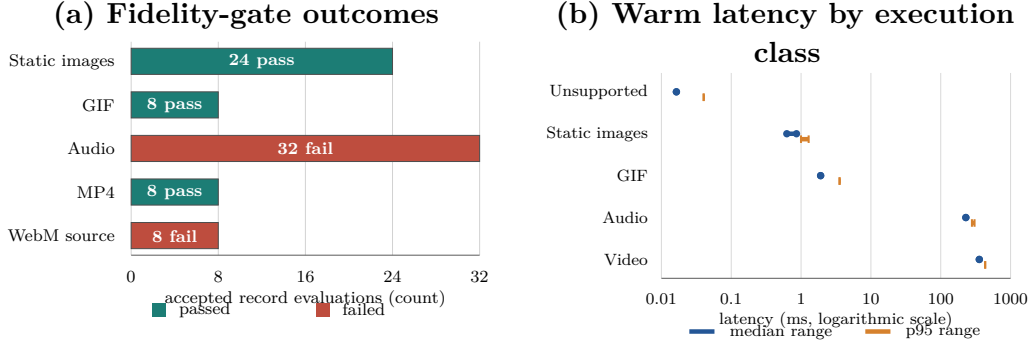


Figure 7: Fidelity outcomes by source or metric group and warm latency by execution class. Panel (a) reports 80 accepted record-level evaluations representing 40 unique output digests. Panel (b) reports the range of profile medians and p95 values within each execution class across 1,560 repeated warm calls, equal to 104 records times five repetitions times three shortened sessions. The logarithmic axis separates immediate rejection, in-process reconstruction, and external codec paths. The observations describe this generated corpus and host only.

tions, representing 40 unique emitted-byte digests, satisfied exact or registered semantic second-pass closure and passed at least two applicable external format/profile checks. Eleven selected single-test fault checks passed. No publication-invariant violation was observed in those 11 executions, and no crash or recorded fuzz failure occurred during the shortened schedule. These are finite-schedule observations, not population failure-rate estimates.

Figure 7 shows two operational limitations. Security acceptance did not imply usability acceptance: all 32 audio and all 8 source-WebM evaluations failed their fidelity profiles, while 24 static-image, 8 GIF, and 8 MP4 evaluations passed. Latency differed by execution path. Unsupported active inputs terminated in tens of microseconds, static images completed below 1 ms at the median, GIF in 1.906 ms, audio near 229 ms, and video near 357 ms. The mixed-corpus warm mean was 125.936 ms/call, corresponding to 7.94 sequential calls/s for this order and host. This is a service rate, not concurrent capacity or deployment throughput.

The action result is a generated-fixture and policy consistency check, not independent ground-truth validation or evidence of source diversity. External format/profile checks corroborate emitted outputs but do not independently validate expected decisions. The experiment therefore supports bounded record-level conformance observations and separately characterizes fidelity and finite-schedule latency for the tested records. The latency figure uses 1,560 repeated warm calls, not independent samples. The shortened schedule supplies neither the prespecified bootstrap intervals nor a controlled RQ3 cost result. The experiment does not isolate the prescan, supplied type evidence, or reconstruction floor because no paired ablation stream was retained.

The absent strata, controlled host, prespecified schedule, immutable external tools, and ablation results keep Equation (32) false and preclude broader efficacy or mechanism claims.

11.1. Public-source compatibility and readiness rehearsals

Table 12 separates the retained observations from the authority they provide. The sequence increases operational readiness for external review but satisfies neither corpus admission nor G_{sci} .

Table 12: Evidence status of the public-source compatibility and readiness rehearsals. Counts are retained observations. The evidence boundary states the maximum supported interpretation for each stage.

Stage	Retained evidence	Evidence boundary
Public-benign pilot	At runner commit 39bf19f1d17b , 10/10 expected-benign records were accepted: 4 byte-idempotent and 6 registered lossy closures. Evidence commit: a03040e	No expected-block record was present, so false acceptance was not estimable. This is finite compatibility evidence only
Candidate inventory	246 Wikimedia Commons and 60 Internet Archive candidates yielded 316 distinct SHA-256 identities totaling 1,569,496,445 bytes. Evidence commit: 1d9e8c931558	The raw shortfall for 246 planned public-source slots fell to zero, but the objects remained candidates rather than audited or admitted corpus records
Mechanical packet	Commit 5153580e428f verified 316 byte identities and retained 948 exact inspection outputs; encoder metadata was observed for 151 candidates	No provenance, license, diversity, or encoder label was independently reviewed. All 316 candidates remained pending and the reviewed-slot deficit was 246
Review and selection	The worklist required 2 matching detached Ed25519 approvals from different reviewer clusters. At selection commit d98b64ae6acc , all 22 cells lacked approved sources and 0/246 sources were selected	No registry, controlled key, signed decision, consensus approval, S0/S1 assignment, full-file admission, or frozen manifest existed; selection failed closed
Offline handoff	Commit 8aa55ef684c2 produced a byte-complete, write-disabled packet with 316 source decision contexts, 722 provider-evidence references, 948 inspection outputs, and a frozen Ed25519 author/sign tool. The 1,534,238,629-byte archive has SHA-256 prefix 35848d3851d6 ; all 208/208 research tests passed. Evidence commit: 8065a35	The packet contains 0 private keys, 0 decisions, and 0 signatures. It makes independent review executable, not completed or approved

12. Discussion

12.1. Answers to the research questions

By definition of the frozen routing relation, RQ1 permits routing exactly when the eligible source-profile set in Equation (6) has cardinality one. Missing or conflicting evidence is not overridden by precedence, and routing success does not authorize a candidate. This is a design-level analytical condition, not an empirical effect.

RQ2 receives a conditional analytical answer within the formal model. Deliverability requires the conjunction in Equation (11), followed by the rule that only a clean result may carry bytes and by the non-overwriting publication transition. The reconstruction order prevents a structural candidate from satisfying a policy that requires semantic reconstruction. Under parser soundness and the stated filesystem assumptions, the conjunction is sufficient for canonical-language membership and non-publication of a new target for every non-clean outcome. Whether the implementation preserves each conjunct is an RQ3 question and is not established by the analytical result.

RQ3 remains unanswered at its full declared scope. The bounded experiment in Section 11 covered 50 S0, 30 S1, and 24 inert S6 generated records plus 11 selected fault checks. Fixture-derived actions, reason families, and pipeline states matched for these records. Forty unique emitted-byte digests passed the applicable external format/profile checks. The experiment did not test the malformed, classifier-disagreement, multi-grammar, resource-exhaustion, or regression strata required by RQ3. All 32 audio and all 8 source-WebM evaluations failed their fidelity profiles. Median latency was below 2 ms for the represented in-process paths and approximately 229–357 ms for the external-codec paths. These finite observations do not meet Equation (32), and the absent paired ablation precludes attribution of a decision to one mechanism.

12.2. Scientific novelty and relation to prior work

Canonical and complete parsing, language-oriented security, content disarm and reconstruction, parser differential analysis, supervised media conversion, and non-overwriting file operations all have established antecedents [4, 5, 7, 8, 10, 17, 18]. Against the selected lines compared in Table 1, the method studied here combines the eight obligations F1–F8 in one authorization relation. Source evidence must select one route. Policy registers the permitted source-to-output conversion and minimum reconstruction level before dispatch. An authorizing parser evaluates the complete candidate on a code path separate from construction. Scanning and resource checks apply to the emitted object, and only a clean result may reach the non-overwriting publication operation. This composition makes structural container closure distinguishable from semantic source-payload independence and prevents failed reconstruction from becoming pass-through delivery. It does not claim that the constituent parser, codec, scanner, or filesystem primitives are individually new.

The resulting security statement is narrower than “malware-free media.” Complete parsing excludes unaccounted candidate prefixes and suffixes under the named grammar. Output evidence convergence makes the generated

name, declared type, byte recognition, and authorizing report agree. On semantic routes, no source payload is copied or referenced as an output component. Under the stated filesystem assumptions, a non-clean result returns no deliverable bytes, creates no new target through the component API, and leaves any pre-existing target unchanged. These properties neither establish universal harmlessness nor rule out unknown decoder flaws.

12.3. Security interpretation

The distinction between structural and semantic reconstruction determines how format coverage may be interpreted. As Table 4 records, structural routes may retain frames, packets, samples, or pixel data while still enforcing container, metadata, and boundary properties. A threat model that includes codec payloads must require semantic reconstruction or block the profile. A single “sanitized” label would conceal this distinction.

The taxonomy in Section 2 supports scenario interpretation but does not determine verdicts. The method measures carrier structure, ambiguity, metadata, resource behavior, reconstruction, and publication. Threat-campaign scenarios connect those observations to realistic delivery chains without inferring malware family or operational role.

12.4. Deployment implications and validity

Deployment begins at the local transition where externally influenced media would otherwise become eligible for storage, decoding, preview, export, or opening by another application. The upstream system may authenticate a server or sender, preserve transport integrity, or record provenance, but those properties do not replace local output authorization. In end-to-end encrypted messaging, this placement also keeps plaintext and verdicts away from a blind relay [1, 2, 3]. In web, mail, cloud, and direct-import settings, the same method applies under different provenance assumptions. Reproducible deployment requires identified parser, policy, and external-tool versions. Different policies may legitimately reject different objects.

The primary internal-validity risk is oracle dependence. Expected actions in the bounded run were generated with the fixtures, so 104/104 agreement measures fixture-policy consistency rather than independent ground truth. A canonical parser can share a misunderstanding with a handler. External tools can also share a codec library. The external checks corroborate emitted format or profile recognition but do not independently validate expected actions or expected blocks. Common-mode faults therefore remain possible. The generated corpus can also overrepresent regular fixtures and underrepresent unusual benign media from specific devices. Grouping descendants

by base artifact prevents direct mutation leakage across partitions, while held-out device and application strata are still needed for external validity.

Fidelity creates another trade-off. Stronger semantic reconstruction reduces source dependence but can change compression artifacts, frame timing, color, audio characteristics, and container features. The security policy and fidelity policy must remain separate so that perceptual similarity does not authorize unsafe output. A profile can be secure under its structural claim and still be operationally unacceptable, or it can satisfy a perceptual metric while failing canonical closure. The G_{sci} gate rejects both cases for different reasons.

Availability claims remain local. The formal model and implementation are designed to bound bytes, dimensions, expansion, elements, deadlines, and supervised child processes for one invocation. The bounded experiment omitted S5 and the complete fault schedule, so it did not empirically establish these bounds. The method also does not provide queue-level fairness, tenant isolation, or network admission control. Memory-safety guarantees narrow the implementation defect surface but do not prove arithmetic, grammar, or semantic correctness.

External-codec supervision likewise does not sandbox arbitrary decoder code. Where decoder compromise is in scope, the semantic routes require the additional platform isolation stated in Table 5. Without it, the article makes no endpoint execution-integrity claim against decoder compromise.

12.5. Evidence state and research directions

Current evidence comprises implementation traceability, a release-ineligible server experiment over 104 generated records, a ten-file public-benign compatibility pilot, and a 316-object candidate inventory. A mechanical packet verified all byte identities and retained 948 inspection outputs. Without an independent registry or signatures, selection returned 0/246 sources across 22 cells. The offline handoff makes review executable but does not create an audited corpus record or support efficacy or release claims.

Next, the write-disabled handoff archive must be delivered outside the audit host to reviewers with distinct key lineages and independence clusters. Matching signed decisions must precede selection, S0/S1 assignment, full-file admission, and a complete S0–S7 manifest. The controlled run must then close the recorded fidelity and format-method gaps and execute the full fuzz, fault, performance, and paired-ablation schedules.

13. Conclusion

Under the registered-parser, implementation-correctness, and filesystem assumptions, the proposed method governs a transport-independent local transition from untrusted media to a deliverable reconstructed object. Delivery requires one source route, a registered source-to-output mapping, policy-sufficient reconstruction, complete authorizing validation on a code path separate from construction, a clean result, and non-overwriting publication. Reconstruction success creates only a candidate. Every other result returns no deliverable bytes, creates no new target through the component interface, and leaves any pre-existing target unchanged.

The analytical model answers RQ1 and RQ2 conditionally. RQ3 remains incomplete. In the bounded experiment, fixture-derived actions, reason families, and pipeline states matched for 104 generated records. Eighty accepted record-level evaluations representing 40 unique output digests passed the applicable external format/profile checks. Eleven selected single-test fault checks found no publication-invariant violation. Seven fuzz targets received 210 requested target-seconds and 7,299,439 iterations without a recorded crash or sanitizer finding. Fidelity failed for all 32 audio and all 8 source-WebM evaluations. The absent S2–S5 and S7 strata, missing paired ablation, shortened schedules, shared host, and failed fidelity gate keep G_{sci} false. A separate ten-file public-benign pilot established only finite compatibility. Mechanical checks covered all 316 candidates, but no independent decision was signed and fail-closed selection returned 0/246 sources. No audited or admitted corpus record was produced. The offline handoff makes review executable but contains no key, decision, or signature; scientific release remains pending.

References

- [1] N. Unger, S. Dechand, J. Bonneau, S. Fahl, H. Perl, I. Goldberg, and M. Smith, "SoK: Secure Messaging," in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 232–249. <https://doi.org/10.1109/SP.2015.22>
- [2] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," *Journal of Cryptology*, vol. 33, no. 4, pp. 1914–1983, 2020. <https://doi.org/10.1007/s00145-020-09360-1>
- [3] H. Abelson, R. Anderson, S. M. Bellovin, J. Benaloh, M. Blaze, J. Callas, W. Diffie, S. Landau, P. G. Neumann, R. L. Rivest, J. I. Schiller, B. Schneier, V. Teague, and C. Troncoso, "Bugs in our pockets: The risks of client-side scanning," *Journal of Cybersecurity*, vol. 10, no. 1, Art. no. tyad020, 2024. <https://doi.org/10.1093/cybsec/tyad020>
- [4] M. Samuel and U. Erlingsson, "Let's Parse to Prevent Pwnage," in *5th USENIX Workshop on Large-Scale Exploits and Emergent Threats*, 2012. <https://www.usenix.org/conference/leet12/workshop-program/presentation/samuel>

- [5] S. Jana and V. Shmatikov, "Abusing file processing in malware detectors for fun and profit," in *2012 IEEE Symposium on Security and Privacy*, 2012, pp. 80–94. <https://doi.org/10.1109/SP.2012.15>
- [6] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson, "The seven turrets of Babel: A taxonomy of LangSec errors and how to expunge them," in *2016 IEEE Cybersecurity Development*, 2016, pp. 45–52. <https://doi.org/10.1109/SecDev.2016.019>
- [7] J. Bangert and N. Zeldovich, "Nail: A practical tool for parsing and generating data formats," in *11th USENIX Symposium on Operating Systems Design and Implementation*, 2014, pp. 615–628. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/bangert>
- [8] T. Ramananandro, A. Delignat-Lavaud, C. Fournet, N. Swamy, T. Chajed, N. Kobeissi, and J. Protzenko, "EverParse: Verified secure zero-copy parsers for authenticated message formats," in *28th USENIX Security Symposium*, 2019, pp. 1465–1482. <https://www.usenix.org/conference/usenixsecurity19/presentation/delignat-lavaud>
- [9] J. Zhang, G. Morrisett, and G. Tan, "Interval parsing grammars for file format parsing," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1073–1095, 2023. <https://doi.org/10.1145/3591264>
- [10] Defense Advanced Research Projects Agency, *Safe Documents (SafeDocs)*, program HR001118S0054, accessed July 12, 2026. <https://www.darpa.mil/research/programs/safe-documents>
- [11] F. Cohen, "Computer viruses: Theory and experiments," *Computers & Security*, vol. 6, no. 1, pp. 22–35, 1987. [https://doi.org/10.1016/0167-4048\(87\)90122-2](https://doi.org/10.1016/0167-4048(87)90122-2)
- [12] E. H. Spafford, "The Internet worm program: An analysis," *ACM SIGCOMM Computer Communication Review*, vol. 19, no. 1, pp. 17–57, 1989. <https://doi.org/10.1145/66093.66095>
- [13] M. Souppaya and K. Scarfone, *Guide to Malware Incident Prevention and Handling for Desktops and Laptops*, NIST Special Publication 800-83 Revision 1, 2013. <https://doi.org/10.6028/NIST.SP.800-83r1>
- [14] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "UNVEIL: A large-scale, automated approach to detecting ransomware," in *25th USENIX Security Symposium*, 2016, pp. 757–772. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/kharaz>
- [15] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and Drop It): Stopping ransomware attacks on user data," in *2016 IEEE 36th International Conference on Distributed Computing Systems*, 2016, pp. 303–312. <https://doi.org/10.1109/ICDCS.2016.46>
- [16] A. Albertini, T. Duong, S. Gueron, S. Kölbl, A. Luykx, and S. Schmieg, "How to abuse and fix authenticated encryption without key commitment," in *31st USENIX Security Symposium*, 2022, pp. 3291–3308. <https://www.usenix.org/conference/usenixsecurity22/presentation/albertini>
- [17] E. Belkind, R. Dubin, and A. Dvir, "Open Image Content Disarm And Reconstruction," arXiv:2307.14057, 2023. <https://doi.org/10.48550/arXiv.2307.14057>
- [18] L. Koch, S. Oesch, A. Sadovnik, B. Weber, A. Chaulagain, M. Dixon, J. Dixon, M. Huettel, C. Watson, J. Hartman, and R. Patulski, "On the abuse and detection of polyglot files," in *Proceedings of the ACM Web Conference 2025*, 2025, pp. 4810–4822. <https://doi.org/10.1145/3696410.3714814>
- [19] Y. You, J. Chen, Q. Wang, and H. Duan, "My ZIP isn't your ZIP: Identifying and exploiting semantic gaps between ZIP parsers," in *34th USENIX Security Symposium*

- sium, 2025, pp. 431–450. <https://www.usenix.org/conference/usenixsecurity25/presentation/you>
- [20] R. Dubin, "Content Disarm and Reconstruction of PDF Files," *IEEE Access*, vol. 11, pp. 38399–38416, 2023. <https://doi.org/10.1109/ACCESS.2023.3267717>
 - [21] R. Fielding, M. Nottingham, and J. Reschke, Eds., *HTTP Semantics*, STD 97, RFC 9110, 2022. <https://doi.org/10.17487/RFC9110>
 - [22] International Telecommunication Union, *Information Technology: Digital Compression and Coding of Continuous-Tone Still Images: Requirements and Guidelines*, Recommendation T.81, 1992. <https://www.itu.int/rec/T-REC-T.81>
 - [23] W3C PNG Working Group, *Portable Network Graphics (PNG) Specification, Third Edition*, W3C Recommendation, June 24, 2025. <https://www.w3.org/TR/png-3/>
 - [24] Google, *WebP Container Specification*, updated August 7, 2025, accessed July 12, 2026. https://developers.google.com/speed/webp/docs/riff_container
 - [25] CompuServe, *Graphics Interchange Format, Version 89a*, 1990. <https://www.w3.org/Graphics/GIF/spec-gif89a.txt>
 - [26] International Organization for Standardization, *Information Technology: Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to about 1.5 Mbit/s, Part 3: Audio*, ISO/IEC 11172-3:1993, 1993. <https://www.iso.org/standard/22412.html>
 - [27] Microsoft, *Resource Interchange File Format (RIFF)*, Win32 documentation, accessed July 20, 2026. <https://learn.microsoft.com/en-us/windows/win32/xaudio2/resource-interchange-file-format--riff->
 - [28] M. Q. C. van Beurden and A. Weaver, *Free Lossless Audio Codec (FLAC)*, RFC 9639, 2024. <https://doi.org/10.17487/RFC9639>
 - [29] S. Pfeiffer, *The Ogg Encapsulation Format Version 0*, RFC 3533, 2003. <https://doi.org/10.17487/RFC3533>
 - [30] Xiph.Org Foundation, *Vorbis I Specification*, July 4, 2020. https://xiph.org/vorbis/doc/Vorbis_I_spec.html
 - [31] T. Terriberry, R. Lee, and R. Giles, *Ogg Encapsulation for the Opus Audio Codec*, RFC 7845, 2016. <https://doi.org/10.17487/RFC7845>
 - [32] International Organization for Standardization, *Information Technology: Coding of Audio-Visual Objects, Part 12: ISO Base Media File Format*, ISO/IEC 14496-12:2022, 2022. <https://www.iso.org/standard/83102.html>
 - [33] MP4 Registration Authority, *Registered Types for the ISO Base Media File Format Family*, accessed July 12, 2026. <https://mp4ra.org/>
 - [34] S. Lhomme, D. Rice, and M. Bunkus, *Extensible Binary Meta Language*, RFC 8794, 2020. <https://doi.org/10.17487/RFC8794>
 - [35] S. Lhomme, M. Bunkus, and D. Rice, *Matroska Media Container Format Specification*, RFC 9559, 2024. <https://doi.org/10.17487/RFC9559>
 - [36] The WebM Project, *WebM Container Guidelines*, accessed July 12, 2026. <https://www.webmproject.org/docs/container/>
 - [37] Matroska Project, *Matroska Version 4 Element Specification*, accessed July 12, 2026. <https://www.matroska.org/technical/elements.html>
 - [38] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 2123–2138. <https://doi.org/10.1145/3243734.3243804>
 - [39] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. <https://doi.org/10.1214/aos/1176344552>